Université de Lorraine - UFR Sciences Fondamentales et Appliquées

Compte rendu du Projet Architecture matérielle pour le format FP8

Par :

Nom : BELKACEM

Prénom : Mohamed Amine

Numéro étudiant : 3252695

Spécialité : Master 1 CSEC

Université de Lorraine – Année 2025-2026

Introduction générale :

L'intelligence artificielle est aujourd'hui un peu partout : reconnaissance d'images, traduction, assistants vocaux, et plus récemment les grands modèles de langage. Derrière tout ça, on retrouve toujours la même brique de base, les réseaux de neurones, qui n'ont fait que grossir au fil des années jusqu'à atteindre plusieurs milliards de paramètres. Le souci, c'est que cette puissance ne sort pas de nulle part : plus un modèle est gros, plus il réclame de mémoire, de bande passante et d'énergie pour tourner.

Une des pistes les plus efficaces pour faire baisser cette facture, c'est de réduire la précision des nombres que la machine manipule. On est ainsi passé du FP32 (32 bits) au FP16 (16 bits) sans perdre grand-chose en qualité, puis on a continué jusqu'au FP8, un format flottant tenant sur seulement 8 bits, proposé en 2022 par NVIDIA, Arm et Intel. Coder un nombre exploitable sur si peu de bits paraît un peu absurde au premier abord, et pourtant ça suffit pour beaucoup d'opérations d'inférence.

Du point de vue matériel, le gain est encore plus parlant : moins de place en mémoire, des bus plus étroits, et surtout des opérateurs arithmétiques bien plus petits sur la puce. Concevoir un circuit capable de représenter et de calculer ces nombres devient alors un vrai sujet d'électronique numérique — et c'est exactement ce qui nous occupe ici.

Problématique. Comment représenter le format FP8 et réaliser une opération arithmétique dessus à l'aide d'une architecture matérielle simple, décrite en VHDL et validée par simulation ?

Objectifs. On s'est fixé trois objectifs. D'abord, comprendre le format FP8 et le situer par rapport au FP32 et au FP16. Ensuite, concevoir une architecture matérielle capable de le manipuler, en partant du multiplieur. Enfin, décrire cette architecture en VHDL, la compiler sous Intel Quartus et la valider en simulation, sans carte réelle. L'idée n'est pas de rivaliser avec les architectures industrielles, mais de construire une version simplifiée et correcte, qu'on puisse réellement réaliser seul dans le cadre d'un projet de semestre.

Organisation du rapport. Le chapitre 1 parle du contexte et l'état de l'art. Le chapitre 2 sur les bases des nombres à virgule flottante. Le chapitre 3 détaille le format FP8 lui-même. Le chapitre 4 décrit l'architecture qu'on a retenue et les choix de conception. Le chapitre 5 est consacré à l'implémentation VHDL et aux résultats de simulation. On termine par une conclusion et quelques perspectives.

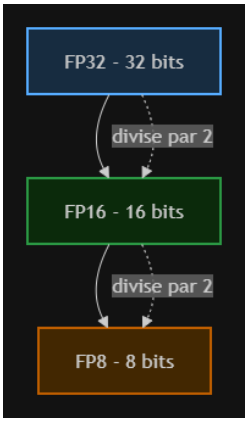


Figure 1 :Évolution des formats à virgule flottante : à chaque étape, la taille est divisée par deux, au prix d'une précision moindre.

1. Contexte général et état de l'art :

Je dois d'abord expliquer pourquoi l'IA pousse à calculer toujours plus, pourquoi ça coûte cher, et comment l'idée de rogner sur la précision des nombres a fini par devenir une vraie solution.

1.1 L'intelligence artificielle et les réseaux de neurones

Depuis le début des années 2010, l'apprentissage profond a complètement rebattu les cartes en intelligence artificielle. Tout a commencer de la reconnaissance d'images : des réseaux de neurones se sont mis à faire mieux que les méthodes classiques, et l'écart n'a fait que se creuser. Le mouvement a ensuite gagné la reconnaissance vocale, la traduction automatique, et plus récemment les grands modèles de langage, ceux qui génèrent du texte.

Derrière toutes ces applications, on retrouve la même brique : le réseau de neurones artificiel. Et quand on regarde ce qu'il y a réellement à l'intérieur, c'est presque décevant de simplicité : un réseau, ce n'est qu'un énorme empilement de multiplications et d'additions sur des matrices de nombres. Ces nombres, ce sont les paramètres du modèle, qu'on appelle aussi les poids. Donc plus un réseau a de poids, plus il est censé être performant.

On est passé de modèles de quelques millions de paramètres à des modèles qui en comptent aujourd'hui des milliards. Or chaque paramètre doit bien être stocké quelque part, puis ressorti et réutilisé à chaque calcul. À cette échelle, ça finit par poser de vrais problèmes côté matériel — et c'est précisément là que notre sujet commence.

1.2 Le coût des calculs : mémoire, bande passante, énergie

Quand on parle du « coût » d'un réseau de neurones, on pense d'abord au temps de calcul. Mais en réalité le problème est plus large, et il touche trois ressources en même temps.

La première, c'est la mémoire. Un modèle d'un milliard de paramètres codés sur 32 bits (4 octets) occupe déjà 4 Go rien que pour ses poids. Il faut pouvoir le stocker, et surtout le charger.

La deuxième, c'est la bande passante. Pendant un calcul, les poids font des allers-retours permanents entre la mémoire et les unités de calcul. Alors que déplacer une donnée coûte souvent plus cher, en temps comme en énergie, que de faire le calcul lui-même. C'est le mur de la mémoire, ce n'est pas le calcul qui bloque, c'est le fait d'alimenter les unités en données assez vite.

La troisième, c'est l'énergie. Dans les grands centres de calcul qui font tourner l'IA, la facture électrique est devenue un facteur de premier plan, pour des raisons de coût comme d'environnement. Réduire l'énergie dépensée par opération est donc passé du statut de détail à celui d'objectif central pour les concepteurs de matériel.

Ces trois contraintes sont liées entre elles : plus les nombres sont gros, plus ils prennent de place, plus ils sont longs à déplacer, et plus ils consomment. D'où l'idée toute bête qui va guider la suite — et si on les rendait simplement plus petits ?

1.3 La réduction de précision comme solution

L'astuce, c'est de représenter les nombres sur moins de bits. Plutôt que de coder chaque valeur sur 32 bits, on essaie 16 bits, puis 8 bits. Cette démarche porte un nom : la réduction de précision (ou quantization dans le cas des entiers).

Pendant longtemps, le standard de l'apprentissage profond a été le flottant 32 bits, le FP32. Puis on s'est aperçu qu'une bonne partie des calculs n'avait pas besoin d'autant de précision. Le format 16 bits, sous ses deux variantes FP16 et BF16, a alors pris le relais, surtout pour l'entraînement. On y perdait un peu de précision, mais sans que la qualité des résultats s'effondre, tout en divisant par deux la mémoire occupée.

La même logique a continué jusqu'au format 8 bits, le FP8, qui est l'objet de ce projet. L'idée a de quoi surprendre : avec 8 bits, on ne dispose que de 256 valeurs distinctes. On voit mal comment ça peut suffire. Et pourtant, pour la phase d'inférence (c'est-à-dire l'utilisation d'un modèle déjà entraîné), cette précision réduite reste souvent acceptable — à condition de bien répartir les bits entre l'exposant et la mantisse. C'est tout l'enjeu du format, et on y reviendra en détail au chapitre 3.

(Voir la figure 1, page précédente, qui illustre cette évolution des formats.)

1.4 Le FP8 dans l'industrie

Le FP8 n'est pas resté une curiosité de labo. Il a été poussé de façon coordonnée par plusieurs gros acteurs du matériel. En 2022, NVIDIA, Arm et Intel ont publié une proposition commune définissant le format FP8 pour l'apprentissage profond. Peu après, l'Open Compute Project a repris ces travaux dans une spécification, ce qui a aidé à standardiser le format au lieu de laisser chaque fabricant bricoler le sien dans son coin.

Côté produits, NVIDIA a intégré le support du FP8 dès sa génération de processeurs graphiques Hopper, justement pensée pour accélérer les calculs d'IA. L'intérêt matériel est double : on gagne sur la mémoire, mais aussi sur la taille des unités de calcul. Un multiplieur qui travaille sur 8 bits est nettement plus petit, plus rapide et moins gourmand qu'un multiplieur 16 ou 32 bits.

Ce raisonnement vaut tout autant pour les FPGA. Comme les ressources logiques d'un FPGA sont limitées, des opérateurs plus petits permettent d'en caser davantage sur la même puce, et donc de paralléliser plus de calculs. C'est précisément ce qui rend le sujet intéressant dans notre cas : on peut décrire une architecture FP8 en VHDL et raisonner concrètement sur son coût matériel.

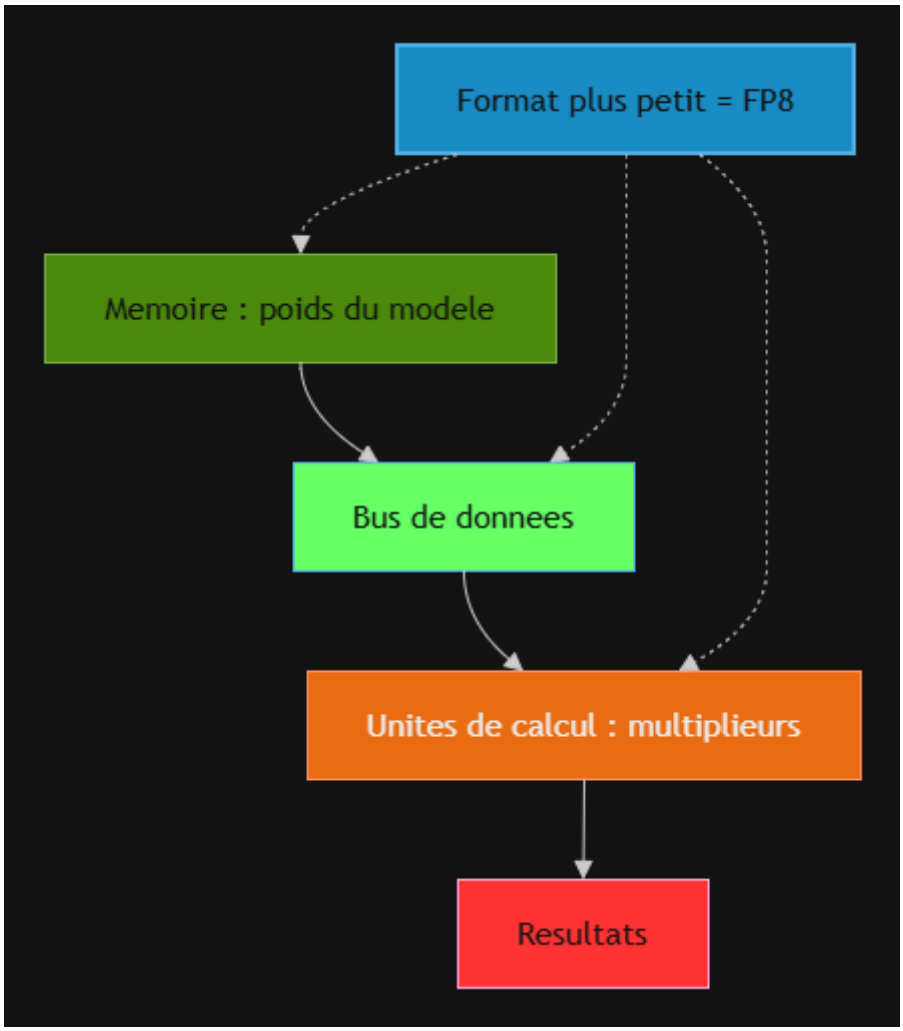


Figure 2 : Un format plus compact (FP8) allège à la fois la mémoire, le bus de données et les unités de calcul

1.5 Positionnement et apport de notre projet

Face à ce contexte industriel, autant être honnête sur ce que notre projet peut apporter. On ne cherche évidemment pas à concurrencer les architectures de NVIDIA ou d'Intel, qui mobilisent des équipes entières sur plusieurs années. Notre but est plus modeste, mais formateur : comprendre le FP8 de l'intérieur et en faire une mise en œuvre matérielle simple, qu'on peut réellement coder, simuler et expliquer soi-même.

On se concentre donc sur un opérateur de base, le multiplieur FP8, qui est aussi l'opération la plus présente dans un réseau de neurones. En le concevant nous-mêmes, on touche à tous les rouages du calcul flottant : décodage des champs, addition des exposants, multiplication des mantisses, normalisation, gestion des cas particuliers. C'est exactement le type de compétence visé par la formation en conception de systèmes électroniques.

Ce chapitre a montré pourquoi le FP8 a vu le jour : l'explosion de la taille des modèles d'IA rend les calculs trop coûteux en mémoire, en bande passante et en énergie, et la réduction de précision est une des réponses les plus efficaces. Le FP8 est aujourd'hui soutenu par les grands fabricants de matériel, ce qui en fait un sujet d'actualité. Le décor étant posé, il faut maintenant reprendre les fondements des nombres à virgule flottante avant d'aborder le FP8 en détail. C'est le rôle du chapitre suivant.

2. Les nombres à virgule flottante

Le format FP8 est un format à virgule flottante. Avant de l'étudier, il faut donc rappeler comment une machine représente les nombres réels, et surtout comment fonctionne la norme qui sert de référence à tous ces formats : l'IEEE 754. Ce chapitre reste volontairement général ; il pose les bases que l'on réutilisera ensuite pour le FP8.

2.1 Représentation des nombres en machine

Un ordinateur ne comprend que en 0 et 1. On doit les coder en binaire. Par exemple, sur 8 bits, on peut représenter les entiers de 0 à 255. Mais des qui il y a un nombre réel ça se complique.

Une première solution serait la virgule fixe : on décide à l'avance combien de bits servent pour la partie entière et combien pour la partie fractionnaire. C'est simple, mais limité. Si on réserve peu de bits à la partie entière, on ne peut pas représenter de grands nombres ; si on en réserve beaucoup, on perd en précision sur les petits nombres.

La virgule flottante résout ce compromis en s'inspirant de la notation scientifique. On ne fixe pas la virgule on la laisse flotter. Un nombre est alors écrit sous la forme d'une mantisse multipliée par une puissance de deux, un peu comme on écrirait $6{,}02 \times 10^{23}$ en notation scientifique décimale. Donc ça nous permet de représenter aussi bien de très petits que de très grands nombres avec le même format.

2.2 La norme IEEE 754

Pour mettre en accord tous les fabriquant et les logiciels sur une même façon de coder les nombres flottants, une norme a été créée : l'IEEE 754. C'est elle qui définit la structure des formats FP32, FP16, et qui sert d'inspiration aux formats plus récents comme le FP8.

Dans cette norme, un nombre flottant est découpé en trois champs :

- le signe (1 bit), qui indique si le nombre est positif ou négatif ;
- l'exposant, qui détermine l'ordre de grandeur du nombre ;
- la mantisse (aussi appelée fraction), qui contient les chiffres significatifs.

La valeur d'un nombre normalisé se calcule avec la formule :

$$valeur = (-1)\text{^}signe \times 1, mantisse \times 2\text{^}(exposant - biais)$$

Le « 1, » placé devant la mantisse est important : comme un nombre normalisé commence toujours par un 1 en binaire, on ne le code pas, il est implicite. C'est ce qu'on appelle le bit caché, et il permet de gagner un bit de précision gratuitement.

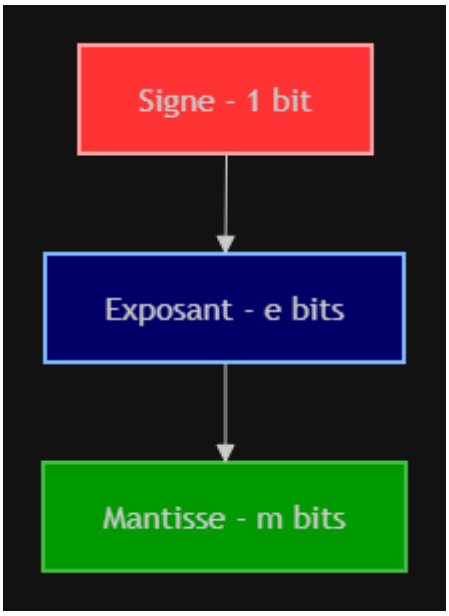


Figure 3 : Structure d'un nombre à virgule flottante.

2.3 Les formats FP32, FP16 et BF16

La norme et ses dérivés se déclinent en plusieurs formats, qui se distinguent surtout par le nombre de bits accordés à l'exposant et à la mantisse. Le tableau suivant résume les principaux.

Tableau 1 : Répartition des bits dans les principaux formats flottants

Format	Total	Signe	Exposant	Mantisse	Biais
FP32	32 bits	1	8	23	127
FP16	16 bits	1	5	10	15
BF16	16 bits	1	8	7	127

Le FP32 est le format historique, très précis mais coûteux. Le FP16 réduit la taille de moitié en rognant surtout sur la mantisse. Le BF16 est plus original : il garde les 8 bits d'exposant du FP32 (donc la même plage de valeurs) mais ne conserve que 7 bits de mantisse. L'idée derrière le BF16, conçu à l'origine par Google pour ses accélérateurs, est qu'en apprentissage profond la plage des valeurs compte souvent plus que la précision. C'est une leçon qu'on retrouvera dans le FP8, qui existe lui aussi en plusieurs variantes selon qu'on privilégie la plage ou la précision.

2.4 Notions clés

Quelques notions reviennent en permanence dès qu'on parle de virgule flottante, et il vaut mieux les avoir bien en tête avant d'attaquer le FP8.

Le biais de l'exposant. L'exposant doit pouvoir être négatif (pour représenter de petits nombres) comme positif (pour de grands nombres). Plutôt que d'utiliser un codage signé classique, la norme ajoute un décalage fixe, le biais. Pour le FP32 par exemple, le biais vaut 127 : un exposant stocké de 130 correspond donc à un exposant réel de 130 − 127 = 3.

Nombres normalisés et sous-normaux. Un nombre normalisé utilise le bit caché à 1 dont on a parlé. Mais pour représenter des valeurs très proches de zéro, la norme prévoit aussi des nombres sous-normaux (ou dénormalisés), où le bit caché vaut 0. Ils améliorent la précision près de zéro, mais compliquent le matériel. On verra que, dans notre projet, on choisira de les ignorer pour rester simple.

L'arrondi. Comme on ne dispose que d'un nombre limité de bits de mantisse, le résultat d'un calcul doit souvent être arrondi. La norme définit plusieurs modes d'arrondi, le plus courant étant l'arrondi au plus proche. Dans une implémentation matérielle simplifiée, on peut aussi se contenter d'une troncature, plus facile à câbler, en assumant la petite perte de précision que cela entraîne.

Les cas particuliers. Enfin, certaines combinaisons de bits sont réservées. Quand l'exposant est entièrement à 0 et la mantisse aussi, le nombre représenté est le zéro. Quand l'exposant est au maximum, on obtient l'infini (mantisse nulle) ou un « NaN », c'est-à-dire « Not a Number » (mantisse non nulle), qui sert à signaler un résultat invalide comme 0/0.

Exemple :

coder 5,0 en FP32.

Prenons le nombre 5,0. En binaire, 5 s'écrit 101, soit $1{,}01 \times 2^2$. On a donc :

signe = 0 (positif) ;

exposant réel = 2, donc exposant stocké = 2 + 127 = 129, soit 10000001 ;

mantisse = 01 suivie de zéros (la partie après le « 1, »).

Au final, 5,0 se code : 0 10000001 01000000000000000000000.

Cet exemple, tout simple, illustre concrètement la formule vue plus haut, et c'est exactement le même raisonnement qu'on appliquera au FP8, juste avec beaucoup moins de bits.

3. Le format FP8 en détail :

Après avoir rappelé les bases du flottant, on peut maintenant entrer dans le vif du sujet. Ce chapitre présente le format FP8, ses deux variantes, leurs particularités, et explique les compromis qu'il impose. C'est aussi ici qu'on commence à préparer les choix qui guideront la conception matérielle du chapitre suivant.

3.1 Présentation du format sur 8 bits

Le FP8 reprend exactement le principe de l'IEEE 754 vu au chapitre précédent : un bit de signe, un champ d'exposant, un champ de mantisse. La seule différence, mais elle est de taille, c'est qu'on ne dispose en tout que de 8 bits. Il faut donc répartir très peu de bits entre l'exposant et la mantisse, et chaque bit compte énormément.

Avec 8 bits, on ne peut représenter que 256 combinaisons différentes. Cela paraît dérisoire comparé aux milliards de valeurs du FP32. Mais l'objectif n'est pas de tout représenter avec précision : il s'agit de couvrir une plage de valeurs suffisante pour les calculs d'IA, avec une précision « juste acceptable ». Tout l'art du format consiste donc à bien équilibrer le partage des bits.

3.2 Les variantes E4M3 et E5M2

Comme un seul partage des bits ne convient pas à tous les usages, le FP8 a été défini en deux variantes, dont les noms décrivent directement la structure :

- E4M3 : 1 bit de signe, 4 bits d'exposant, 3 bits de mantisse. Le biais vaut 7.
- E5M2 : 1 bit de signe, 5 bits d'exposant, 2 bits de mantisse. Le biais vaut 15.

La logique est simple. En donnant un bit de plus à l'exposant, le E5M2 couvre une plage de valeurs plus large, mais avec moins de précision (seulement 2 bits de mantisse). À l'inverse, le E4M3 sacrifie un peu de plage pour gagner en précision. En pratique, on utilise plutôt le E4M3 pour les poids et les activations d'un réseau (où la précision compte), et le E5M2 pour les gradients pendant l'entraînement (où il faut surtout une grande plage).

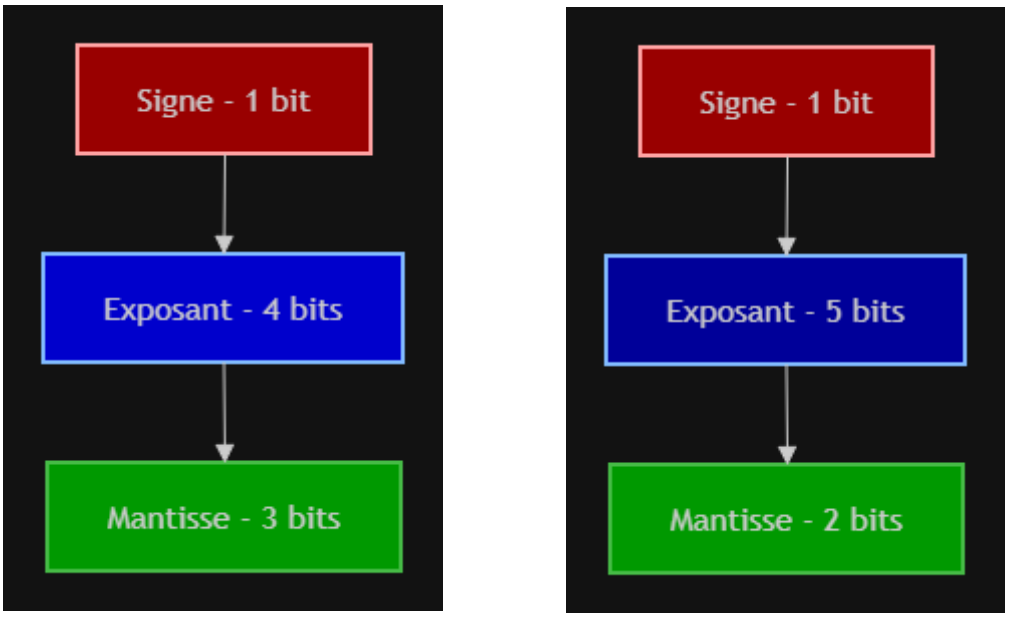


Figure 4 : à gauche le format E4M3, a droite le format E5M2. Le E5M2 donne un bit de plus à l'exposant au détriment de la mantisse.

3.3 Plage dynamique, précision et encodages particuliers

Une fois la structure connue, on peut calculer ce que chaque variante est capable de représenter.

Pour le E4M3, la plus grande valeur représentable est 448, et la plus petite valeur normale vaut environ 0,0156 (soit 2^{-6}). Le format ne définit pas l'infini ; seule une combinaison particulière (tous les bits d'exposant à 1 et tous ceux de mantisse à 1) est réservée pour signaler un NaN. Cela laisse donc un peu plus de codes disponibles pour les nombres utiles.

Pour le E5M2, la plus grande valeur grimpe jusqu'à 57344, grâce au bit d'exposant supplémentaire. En contrepartie, avec seulement 2 bits de mantisse, les valeurs représentables sont beaucoup plus espacées. Cette variante suit de plus près l'IEEE 754 et conserve, elle, l'infini et le NaN.

Dans les deux cas, le zéro est codé comme en IEEE 754, par un exposant et une mantisse tous nuls. Et comme pour les autres formats, des nombres sous-normaux existent pour les valeurs très proches de zéro, même si on choisira plus loin de ne pas les implémenter.

3.4 Comparaison FP32 / FP16 / FP8

Pour bien situer le FP8 par rapport aux formats déjà vus, le tableau ci-dessous rassemble l'essentiel.

Tableau 2 : Comparaison des formats à virgule flottante.

Format	Total	Exp.	Mant.	Biais	Valeur max	Usage typique
FP32	32	8	23	127	$\approx 3{,}4 \times 10^{38}$	référence, haute précision
FP16	16	5	10	15	65504	entraînement, inférence
FP8 E4M3	8	4	3	7	448	poids, activations
FP8 E5M2	8	5	2	15	57344	gradients

Ce tableau résume bien le compromis général : à chaque fois qu'on enlève des bits, on gagne en compacité matérielle mais on perd soit en plage, soit en précision. Le FP8 pousse ce compromis à son maximum, ce qui le rend très efficace côté matériel, mais beaucoup plus délicat à utiliser sans dégrader les résultats

Exemple :
coder 6,0 en E4M3.

Reprenons la même démarche qu'au chapitre 2, mais avec le format qu'on va implémenter. Le nombre 6 s'écrit 110 en binaire, soit $1{,}10 \times 2^2$. On a donc :

- signe = 0 ;
- exposant réel = 2, donc exposant stocké = 2 + 7 = 9, soit 1001 ;
- mantisse = 100 (les chiffres après le « 1, », complétés sur 3 bits).

Au final, 6,0 se code 0 1001 100 en E4M3. On peut vérifier : $1{,}100_2 \times 2^(9-7) = 1{,}5 \times 4 = 6$. Cet exemple montre aussi une limite du format : entre deux valeurs codables, l'écart est important. Avec seulement 3 bits de mantisse, on ne peut pas représenter n'importe quel nombre, et le résultat des calculs devra souvent être arrondi.

3.5 Avantages et limites du FP8 :

Les avantages du FP8 découlent directement de sa petite taille. Côté mémoire, il divise par deux l'occupation par rapport au FP16, et par quatre par rapport au FP32. Côté matériel, les opérateurs sont bien plus petits : un multiplieur FP8 mobilise beaucoup moins de logique qu'un multiplieur FP16. Cela se traduit par des circuits plus rapides, moins gourmands en énergie, et qui tiennent en plus grand nombre sur une même puce ou un même FPGA.

Mais ces avantages ont une contrepartie. La très faible précision impose des précautions : on ne peut pas utiliser le FP8 partout sans risquer de dégrader les résultats. En pratique, il est souvent réservé à certaines parties d'un calcul, le reste restant dans un format plus précis. C'est d'ailleurs pour cela

qu'il existe deux variantes : on adapte le format à l'usage. Pour notre projet, ces limites ne sont pas un obstacle, car notre but est de comprendre et de réaliser l'opérateur, pas d'optimiser un réseau complet.

4 Conception et réalisation de l'architecture matérielle FP8

Après avoir compris le format FP8, on peut passer à la conception de l'opérateur, et surtout à sa réalisation en VHDL sous Quartus. J'ai choisit de réaliser un multiplieur-accumulateur (MAC), plus représentatif des calculs d'intelligence artificielle. Dans ce chapitre, chaque choix de conception que j'ai fait est suivi de sa traduction en VHDL, bloc par bloc, du décodage jusqu'à l'accumulateur. L'objectif est que l'architecture soit correcte, et qu'elle reste assez simple pour être codée et simulée seul.

4.1 Choix du format et hypothèses de conception

Le premier choix concerne la variante du FP8. Nous avons retenu le E4M3 (1 bit de signe, 4 bits d'exposant, 3 bits de mantisse, biais 7), la plus utilisée pour les poids et les activations, et dont les 3 bits de mantisse rendent les résultats plus lisibles en simulation.

Le deuxième choix concerne l'opération : un MAC, qui calcule accumulateur = accumulateur ± (a × b). C'est exactement l'opération de base d'un neurone, et elle nous fait couvrir la multiplication, l'addition et la logique séquentielle d'un seul coup.

Enfin, pas de sous-normaux (les valeurs trop petites deviennent zéro), arrondi par troncature, saturation à 448 en cas de dépassement, retour à zéro en cas de soupassement, et accumulation entièrement en FP8 .

Le package fp8_pkg regroupe les paramètres du format pour ne pas les réécrire partout, notamment le biais :

```
constant EXP_BIAS : integer := 7;  -- biais de l'exposant E4M3
```

Ce simple choix d'un package partagé évite les erreurs et rend le code plus propre.

4.2 Architecture globale du MAC

Le MAC enchaîne deux opérations et mémorise le résultat. À chaque coup d'horloge actif, il multiplie les deux entrées, ajoute (ou retranche) le produit à la valeur accumulée, et range le tout dans un registre. On le décompose donc en quatre blocs : le multiplieur, l'additionneur/soustracteur, l'accumulateur, et la logique de contrôle (horloge, reset, activation, effacement, choix addition/soustraction).

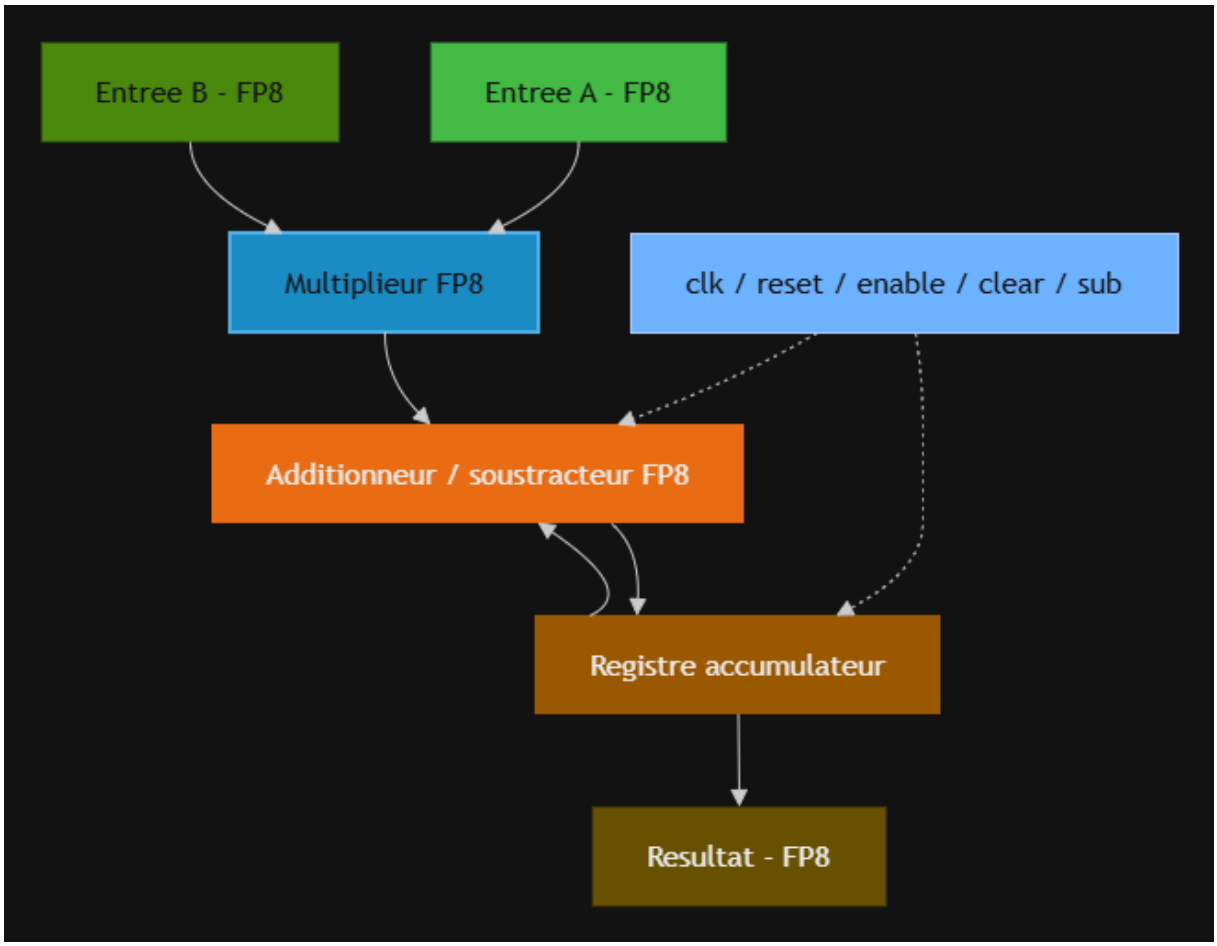


Figure 5 : Architecture globale du MAC FP8.

fp8_mac expose toutes les commandes du schéma et instancie les trois sous-modules relié par des signaux internes :

```
entity fp8_mac is
    port (
        clk, reset, clear, enable, sub : in  std_logic;
        a, b    : in  std_logic_vector(7 downto 0);
        result : out std_logic_vector(7 downto 0);
        valid  : out std_logic
    );
end entity fp8_mac;
```

J’ai testé chaque bloc séparément avant d'être assemblé.

4.3 Le décodage et l'encodage du FP8

Pour décoder un nombre FP8 on doit extraire ses trois champs : sur 8 bits numérotés de 7 à 0, le bit 7 est le signe, les bits 6 à 3 l'exposant, et les bits 2 à 0 la mantisse. Il n'y a aucun calcul, il y a qu'un découpage, ce qui se traduit en VHDL dans le module fp8_decoder :

```
sign     <= fp8_in(7);
exponent <= fp8_in(6 downto 3);
mantissa <= fp8_in(2 downto 0);
is_zero  <= '1' when (fp8_in(6 downto 0) = "0000000") else '0';
```

Le signal is_zero détecte le cas particulier du zéro, repéré quand les 7 bits de poids faible sont tous nuls. Ce décodeur est codé et testé à part, pour valider l'extraction des champs avant de s'en servir dans les calculs. À l'inverse, l'encodage est trivial : il suffit de concaténer signe, exposant et mantisse, opération qu'on retrouve à la fin de chaque module sous la forme signe & exposant & mantisse.

4.4 Le multiplieur FP8

Le multiplieur reprend la multiplication en notation scientifique, l'opération la plus simple à câbler en flottant car elle n'exige aucun alignement. Son algorithme : signe par XOR, reconstruction des significandes avec le bit caché, multiplication des significandes, addition des exposants moins le biais, puis normalisation et troncature.

En VHDL, c'est le module fp8_mult (entrées a, b, sortie p). Le bit caché s'ajoute simplement en collant un '1' devant la mantisse, ce qui donne un significande de 4 bits :

```
sig_a := unsigned('1' & a(2 downto 0));   -- significande 1.xxx
sig_b := unsigned('1' & b(2 downto 0));
prod  := sig_a * sig_b;                    -- 4x4 = 8 bits
```

L'exposant se calcule par ea + eb - EXP_BIAS, et la normalisation consiste à regarder si le produit a dépassé 2 (c'est-à-dire si son bit de poids fort est à 1) pour décaler et ajuster l'exposant :

```
if prod(7) = '1' then                  -- produit >= 2.0
    man_res := std_logic_vector(prod(6 downto 4));
    exp_res := exp_sum + 1;
else
    man_res := std_logic_vector(prod(5 downto 3));
    exp_res := exp_sum;
end if;
```

On voit ici concrètement la troncature : on ne garde que trois bits de mantisse et on jette le reste, sans arrondi. C'est un choix assumé pour simplifier le circuit.

4.5 L'additionneur/soustracteur FP8

C'est le module le plus délicat, car il faut d'abord aligner les deux nombres avant de pouvoir les combiner. Nous l'avons conçu comme un additionneur/soustracteur : un signal sub permet de retrancher au lieu d'ajouter, simplement en inversant le bit de signe de l'opérande, puisque soustraire revient à ajouter l'opposé.

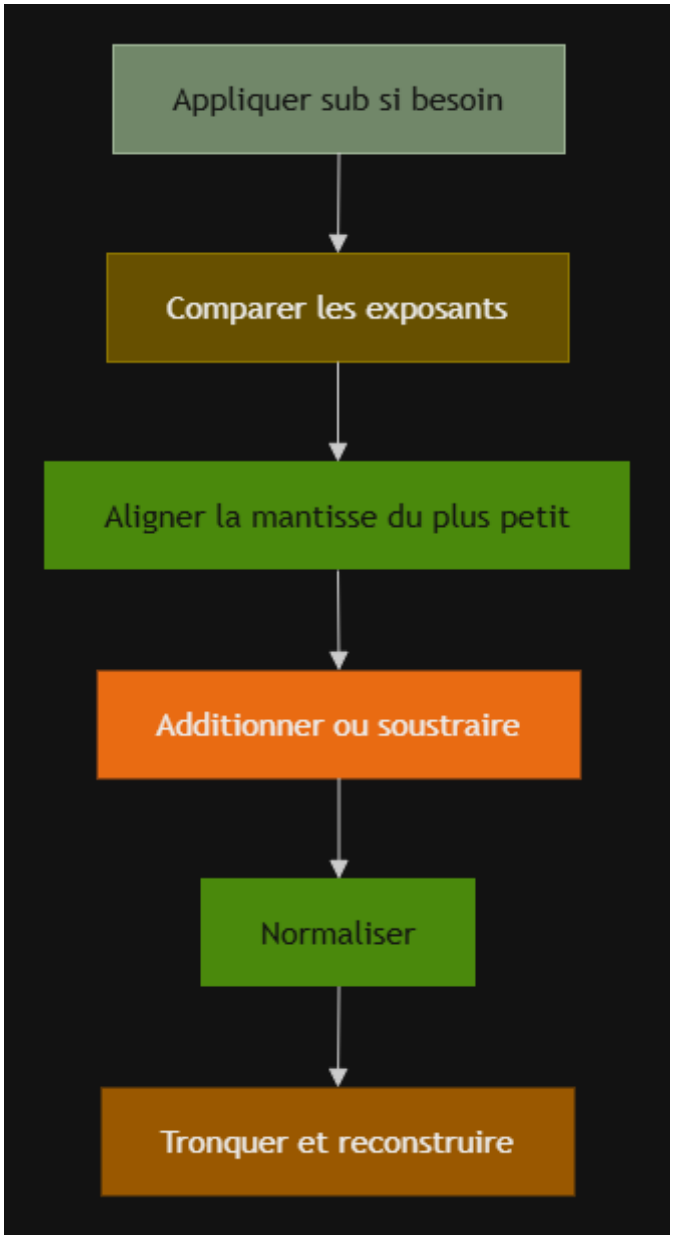


Figure 6 : Étapes de l'additionneur/soustracteur.

En vert, les deux étapes de décalage propres au flottant

```
bb := b;
if sub = '1' then bb(7) := not b(7); end if;
```

L'alignement, lui, consiste à décaler la mantisse du plus petit nombre vers la droite, du nombre de positions égal à la différence des exposants :

Vhdl

```
sml_ext := shift_right(sml_ext, exp_diff);
```

Après l'addition (ou la soustraction) des significandes, la normalisation ramène le résultat sous la forme 1,xxx. C'est ici qu'apparaît la subtilité du flottant : en cas de retenue on décale d'un bit à droite, mais en cas de simplification entre deux nombres proches il faut décaler à gauche, parfois de plusieurs positions, ce qu'on fait par une boucle qui ajuste l'exposant à chaque pas. Ces décalages, qui n'existent pas dans le multiplieur, sont exactement ce qui rend l'additionneur plus complexe.

4.6 L'accumulateur synchrone et les signaux de contrôle

Jusqu'ici, multiplieur et additionneur sont purement combinatoires : ils n'ont pas de mémoire. Pour accumuler, il faut au contraire conserver le résultat d'un cycle au suivant, et c'est le rôle de l'accumulateur, un registre synchronisé sur l'horloge.

Concrètement, le module fp8_acc charge la sortie de l'additionneur à chaque front montant si l'activation est présente, avec un reset asynchrone et un effacement synchrone :

```
process(clk, reset)
begin
    if reset = '1' then
        reg <= (others => '0');
    elsif rising_edge(clk) then
        if clear = '1' then
            reg <= (others => '0');      -- nouvelle accumulation
        elsif enable = '1' then
            reg <= d_in;                 -- on accumule
        end if;
    end if;
end process;
```

C'est ce bloc séquentiel qui transforme une suite de multiplications et d'additions en un véritable opérateur capable d'enchaîner les calculs. Le signal clear permet de repartir de zéro pour une nouvelle série, et un signal valid, généré dans le top-level, signale qu'une nouvelle valeur accumulée est disponible.

4.7 Gestion des cas particuliers

Plusieurs situations doivent être traitées à part. Le zéro est détecté en entrée : si l'une des entrées du multiplieur est nulle, le produit est nul ; et dans l'additionneur, si un opérande est nul, le résultat est directement l'autre.

Le dépassement survient quand l'exposant d'un résultat dépasse le maximum. Plutôt que de produire l'infini, qu'on ne gère pas, on sature à la plus grande valeur (448), en conservant le signe. En VHDL, cette saturation et le soupassement se règlent au moment de l'assemblage du résultat :

```
if exp_res > 15 then
    s <= sbig & "1111" & "110";     -- saturation a 448
elsif exp_res < 1 then
    s <= (others => '0');           -- soupassement -> 0
else
    s <= sbig & std_logic_vector(to_unsigned(exp_res, 4)) & man_res;
end if;
```

Ces trois cas seront vérifiés explicitement en simulation, car ce sont eux qui prouvent que l'opérateur se comporte correctement aux limites du format.

5 Implémentation VHDL et simulation sous Quartus

Une fois tous les modules conçus, il a fallu passer à la partie la plus concrète du projet : écrire le code, le compiler et surtout vérifier qu'il fonctionne vraiment. Ce chapitre raconte cette étape. On commence par présenter l'environnement de travail, puis on décrit chaque module un par un, en montrant à chaque fois le schéma matériel généré par l'outil et les ressources qu'il consomme, du plus simple au plus complexe. On présente ensuite les bancs de test et les résultats de simulation, avant de terminer par une analyse globale. L'idée est de prouver que l'opérateur ne reste pas une idée sur le papier : il devient un vrai circuit, et il se comporte comme prévu.

5.1 Environnement de travail

Tout le projet a été réalisé sous Intel Quartus Prime 25.1 (édition Standard), l'outil de référence pour la conception sur cible Intel/Altera. Pour la simulation, nous avons utilisé Questa Altera Starter FPGA Edition, le simulateur livré avec Quartus. Comme le sujet le demandait, aucune carte physique n'a été utilisée : le travail s'est arrêté à l'écriture du VHDL, à la compilation et à la simulation fonctionnelle, dite simulation RTL.

Les deux outils ne servent pas à la même chose, et nous les avons utilisés séparément du début à la fin. Quartus s'occupe de la synthèse : il transforme le code en circuit logique réel et calcule les ressources que ce circuit occuperait sur une puce ; c'est lui qui produit le schéma RTL et le rapport de ressources (le Flow Summary). Questa, lui, s'occupe de la simulation : il applique des valeurs aux entrées et affiche les sorties sous forme de chronogrammes. À noter que les bancs de test ne servent qu'à Questa, jamais à la synthèse, puisqu'ils ne décrivent pas un circuit mais seulement un scénario de vérification.

Pour rester organisé, tout le projet a été regroupé dans un seul dossier, avec un package de paramètres, cinq modules et quatre bancs de test. Cette organisation simple a beaucoup aidé pendant la mise au point, car elle permet de compiler et de tester chaque module séparément avant de tout assembler.

5.2 Description des modules : architecture et ressources

Cette section décrit les cinq modules un par un. Pour chacun, nous donnons son rôle, puis nous montrons deux résultats fournis par Quartus après synthèse : le schéma RTL, qui prouve que le code se traduit bien en un circuit matériel, et le Flow Summary, qui indique les ressources logiques occupées. Nous procédons du module le plus simple, le décodeur, jusqu'au plus complexe, le MAC, qui rassemble tous les autres.

Avant d'entrer dans le détail, le tableau suivant récapitule les modules.

Tableau 3 : Les modules VHDL du projet et leurs interfaces.

Module	Type	Rôle	Entrées	Sorties
fp8_pkg	package	Paramètres du format	—	—
fp8_decoder	combinatoire	Sépare signe/exposant/mantisse	fp8_in	sign, exponent, mantissa, is_zero
fp8_mult	combinatoire	Multiplie deux FP8	a, b	p
fp8_addsub	combinatoire	Additionne ou soustrait	a, b, sub	s
fp8_acc	séquentiel	Mémorise l'accumulateur	clk, reset, clear, enable, d_in	q_out
fp8_mac	top-level	Multiplie-accumule	clk, reset, clear, enable, sub, a, b	result, valid

5.2.1 Le décodeur (fp8_decoder)

Le décodeur est le module le plus simple. Son rôle est d'ouvrir un nombre FP8 pour en extraire les trois informations qu'il contient : le signe, l'exposant et la mantisse. Sans cette étape, aucun calcul n'est possible, car le multiplieur comme l'additionneur ont besoin de manipuler ces champs séparément. Le décodeur produit en plus un quatrième signal, is_zero, qui indique si le nombre présenté est nul. Ce signal est utile car le zéro est un cas particulier en virgule flottante : il ne suit pas la formule habituelle et doit être traité à part.

Le fonctionnement repose entièrement sur la façon dont les bits sont rangés dans le format E4M3. Sur les 8 bits numérotés de 7 à 0, le bit 7 porte le signe, les bits 6 à 3 forment l'exposant et les bits 2 à 0 forment la mantisse. Décoder revient donc simplement à « couper » le mot binaire aux bons endroits et à diriger chaque morceau vers la bonne sortie. Il n'y a ni addition, ni multiplication, ni mémorisation : c'est une opération purement combinatoire, où les sorties suivent instantanément l'entrée.

La vue RTL générée par Quartus illustre parfaitement ce comportement. On y voit l'entrée fp8_in[7..0] arriver à gauche, puis se diviser en plusieurs faisceaux de fils. Un fil isolé, le bit 7, part directement vers la sortie sign. Un groupe de bits, repéré « 6:3 », est dirigé vers exponent[3..0]. Un autre groupe, « 2:0 », est dirigé vers mantissa[2..0]. Sur ces trois chemins, il n'y a aucune porte logique : ce ne sont que des connexions, ce qui confirme que le décodage est un simple réacheminement des bits, exactement comme prévu dans la conception. Le seul véritable composant de calcul présent sur le schéma est le bloc repéré « Equal0 », un comparateur. Celui-ci prend les 7 bits de poids faible de l'entrée et les compare à la valeur zéro ; sa sortie pilote le signal is_zero. C'est la traduction matérielle directe de la condition que nous avions écrite en VHDL pour détecter le zéro. Ce schéma est donc une preuve concrète que notre description se transforme bien en un circuit, et non en un programme.

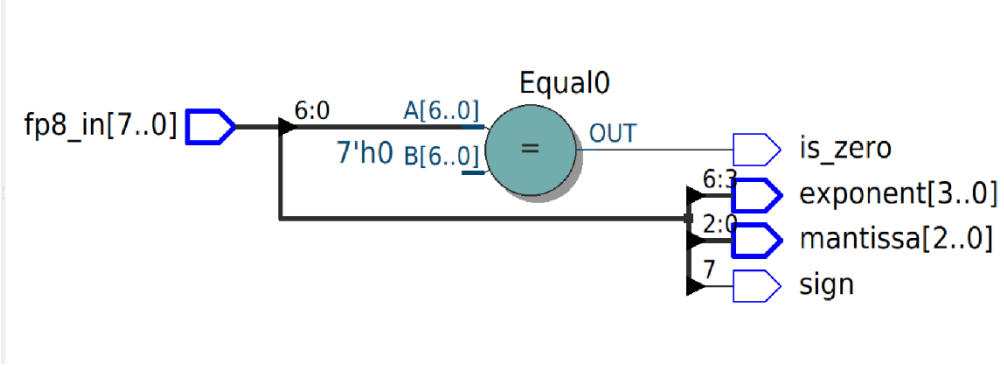
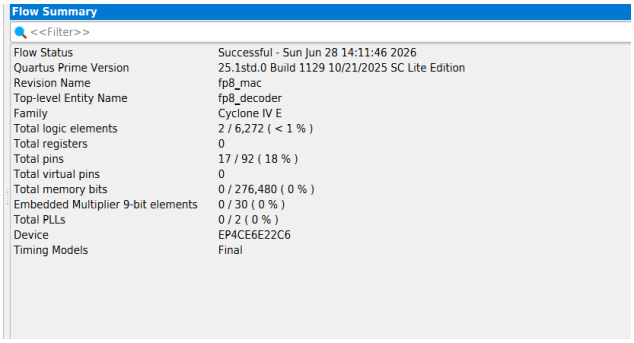


Figure 7 : vue RTL du décodeur

Le rapport de synthèse, ou Flow Summary, vient confirmer chiffre à l'appui cette grande simplicité. Le décodeur n'occupe que 2 éléments logiques sur les 6 272 disponibles dans le composant Cyclone IV E retenu, ce qui représente moins de 1 % des ressources. Il n'utilise aucun registre, ce qui est cohérent avec sa nature combinatoire : comme il ne mémorise rien d'un instant à l'autre, il n'a pas besoin de bascules. On relève également l'utilisation de 17 broches, qui correspondent tout simplement aux fils d'entrée et de sortie du module (l'entrée sur 8 bits, plus les différentes sorties). Aucun multiplieur matériel ni bloc mémoire n'est mobilisé. Ce coût quasiment nul n'est pas surprenant : extraire les champs d'un nombre ne demande pas de logique, juste du câblage. Il illustre néanmoins un point important pour la suite, à savoir que les opérations « de préparation » comme le décodage sont pratiquement gratuites en matériel, et que ce sont les vrais calculs (multiplication, addition) qui consommeront l'essentiel des ressources.


Flow Summary

<<Filter>>

Flow Status	Successful - Sun Jun 28 14:11:46 2026
Quartus Prime Version	25.1std.0 Build 1129 10/21/2025 SC Lite Edition
Revision Name	fp8_mac
Top-level Entity Name	fp8_decoder
Family	Cyclone IV E
Total logic elements	2 / 6,272 (< 1 %)
Total registers	0
Total pins	17 / 92 (18 %)
Total virtual pins	0
Total memory bits	0 / 276,480 (0 %)
Embedded Multiplier 9-bit elements	0 / 30 (0 %)
Total PLLs	0 / 2 (0 %)
Device	EP4CE6E22C6
Timing Models	Final

Figure 8 : Flow Summary du décodeur

5.2.2 Le multiplieur (fp8_mult)

Le multiplieur est le premier vrai opérateur de calcul de notre architecture, et aussi le cœur du MAC. Sa tâche est de multiplier deux nombres FP8 et de fournir le résultat, lui aussi en FP8. Contrairement au décodeur, il ne se contente pas de réorganiser des bits : il effectue de véritables opérations arithmétiques. Pour rappel, multiplier deux flottants revient à multiplier leurs significandes, additionner leurs exposants, puis normaliser et arrondir le résultat. C'est exactement cet enchaînement que l'on retrouve dans le circuit synthétisé.

La vue RTL est nettement plus riche que celle du décodeur, ce qui est attendu puisque le module fait de l'arithmétique. En la parcourant de gauche à droite, on peut reconnaître les grandes étapes de l'algorithme. On distingue d'abord un bloc multiplieur, repéré « Mult0 », qui prend les significandes des deux entrées (chacune sur 4 bits, bit caché compris) et calcule leur produit. On voit ensuite plusieurs additionneurs, repérés « Add0 », « Add1 » et « Add2 » : ce sont eux qui réalisent l'addition des exposants et le retrait du biais. La présence de la constante « 7'h71 » sur l'une des entrées d'addition correspond justement à la prise en compte du biais dans le calcul de l'exposant. Plus à droite, on remarque des comparateurs, notés « LessThan0 » et « LessThan1 », qui servent à détecter les cas de soupassement et de dépassement de l'exposant, ainsi que des multiplexeurs (les blocs en forme de trapèze, comme « man_res » et « exp_res ») qui choisissent la bonne valeur de mantisse et d'exposant selon que le produit a dû être normalisé ou non. Tout en bas, on retrouve deux comparateurs « Equal0 » et « Equal1 » reliés à une porte logique « process_0 » : c'est la détection des entrées nulles, qui force le résultat à zéro lorsque l'un des opérandes est nul. Enfin, une porte « sp » (un XOR) calcule le signe du produit à partir des deux signes d'entrée. Ce schéma est donc la traduction fidèle, en portes et en blocs, de l'algorithme que nous avions décrit étape par étape au chapitre 4.

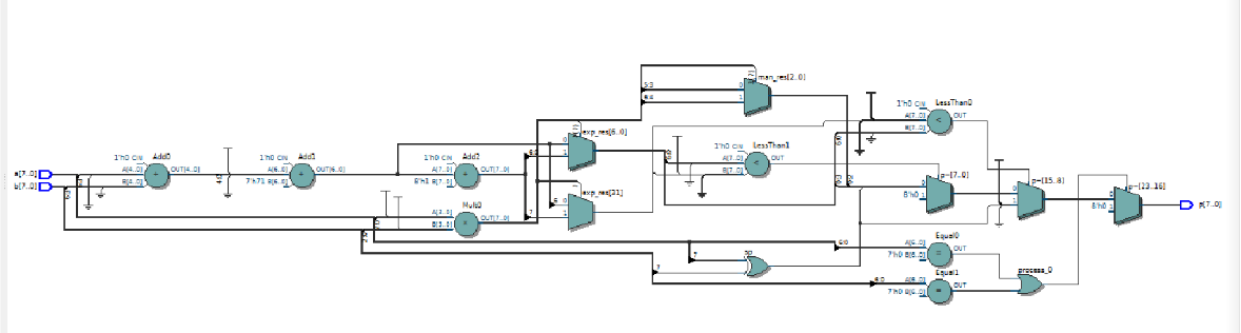


Figure 9 : vue RTL globale du multiplieur

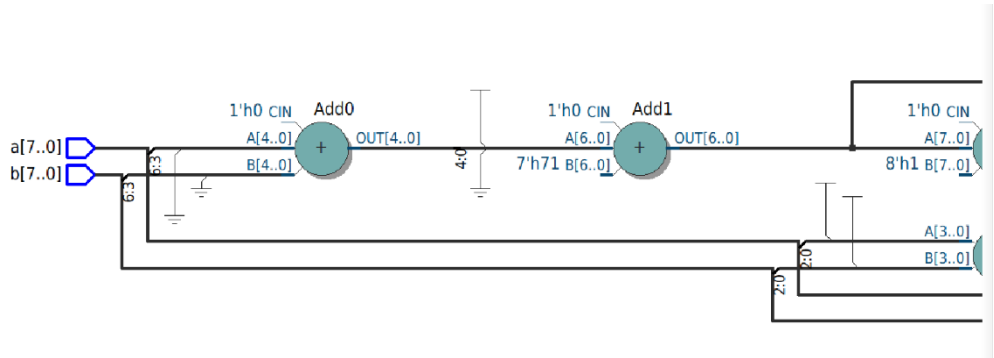


Figure 10 : vue RTL de l'entrée du multiplieur

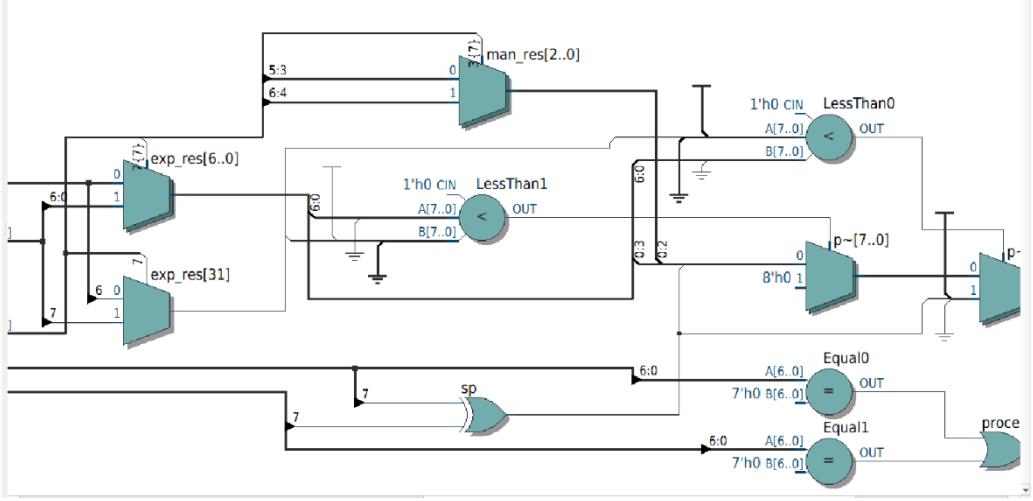


Figure 11 : vue RTL plus détailler du multiplieur

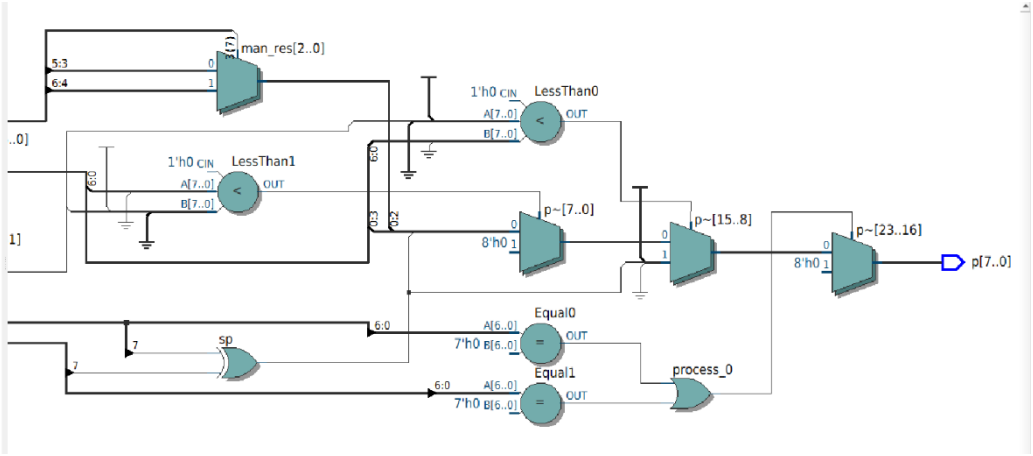


Figure 12 : vue RTL de la sortie du multiplieur

Le Flow Summary chiffre le coût de ce circuit. Le multiplieur utilise 66 éléments logiques sur les 6 272 disponibles, soit environ 1 % du composant, et toujours aucun registre, ce qui confirme qu'il s'agit d'un circuit purement combinatoire. On notera un point intéressant : la ligne « Embedded Multiplier 9-bit elements » indique 0, ce qui signifie que Quartus n'a pas eu besoin d'utiliser les multiplieurs matériels dédiés du FPGA. La multiplication des significandes, qui ne porte que sur 4 bits, est si petite qu'elle est réalisée directement avec de la logique ordinaire. C'est une bonne illustration de l'intérêt du FP8 : même l'opération la plus coûteuse, la multiplication, reste minuscule lorsqu'elle travaille sur si peu de bits. On comprend mieux, à travers ce chiffre, pourquoi le format est si attractif pour le matériel. En comparant avec le décodeur (2 éléments logiques), on voit nettement où se situe le vrai coût : ce n'est pas la préparation des données, mais bien le calcul lui-même.

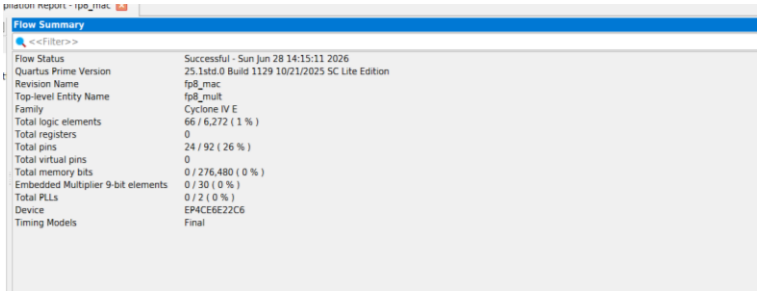


Figure 13 : Flow Summary du multiplieur

5.2.3 L'additionneur/soustracteur (fp8_addsub) :

L'additionneur/soustracteur est le module le plus complexe de toute l'architecture, et sa vue RTL le montre clairement : le schéma est nettement plus dense que celui du multiplieur. Cette complexité n'est pas un défaut, elle est inhérente à l'addition flottante. Comme nous l'avons expliqué au chapitre 4, additionner deux flottants oblige d'abord à aligner leurs exposants avant de pouvoir combiner les mantisses, puis à renormaliser le résultat, qui peut nécessiter un décalage important. Chacune de ces étapes se traduit par de la logique supplémentaire, d'où la taille du circuit.

En examinant le schéma, on retrouve les différentes phases de l'algorithme. À l'entrée, on remarque la gestion du signal sub : le bit de signe de l'opérande b peut être inversé, ce qui permet de transformer une addition en soustraction sans circuit dédié. On repère ensuite plusieurs comparateurs (« LessThan » et « Equal ») qui servent à déterminer lequel des deux nombres est le plus grand et à détecter les cas particuliers comme les entrées nulles. Le cœur de l'alignement apparaît sous la forme d'un bloc de décalage repéré « ShiftRight0 » : c'est lui qui décale la mantisse du plus petit nombre pour la mettre à la même échelle que l'autre, du nombre de positions correspondant à la différence des exposants. On distingue aussi plusieurs additionneurs (« Add0 » à « Add5 ») qui effectuent les calculs sur les exposants et les mantisses, ainsi que de nombreux multiplexeurs (les blocs trapézoïdaux comme « big_sig », « sml_sig », « sbig », « ssml », « ebig », « exp_diff ») qui aiguillent les bonnes valeurs selon que a ou b est le plus grand. Enfin, toute une cascade de multiplexeurs en sortie (« s~[7..0] », « s~[15..8] », etc.) reconstitue le résultat final après normalisation. Ce foisonnement de blocs reflète directement les nombreuses étapes conditionnelles de l'addition flottante.

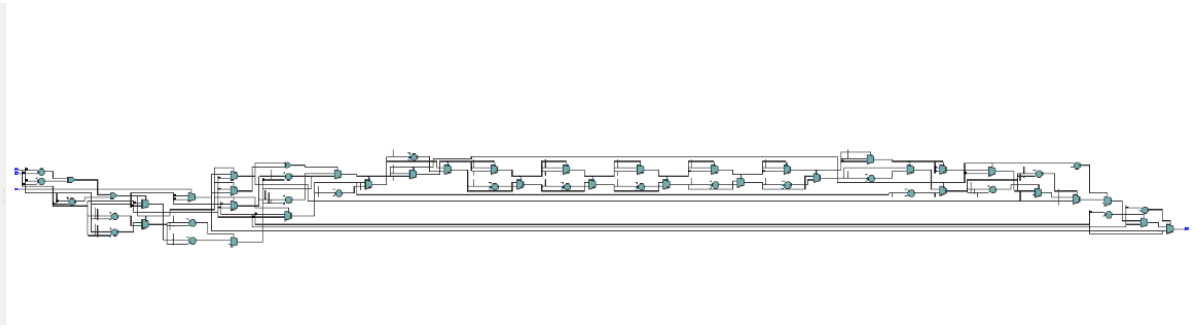


Figure 14 : vue RTL globale de l'additionneur/soustracteur

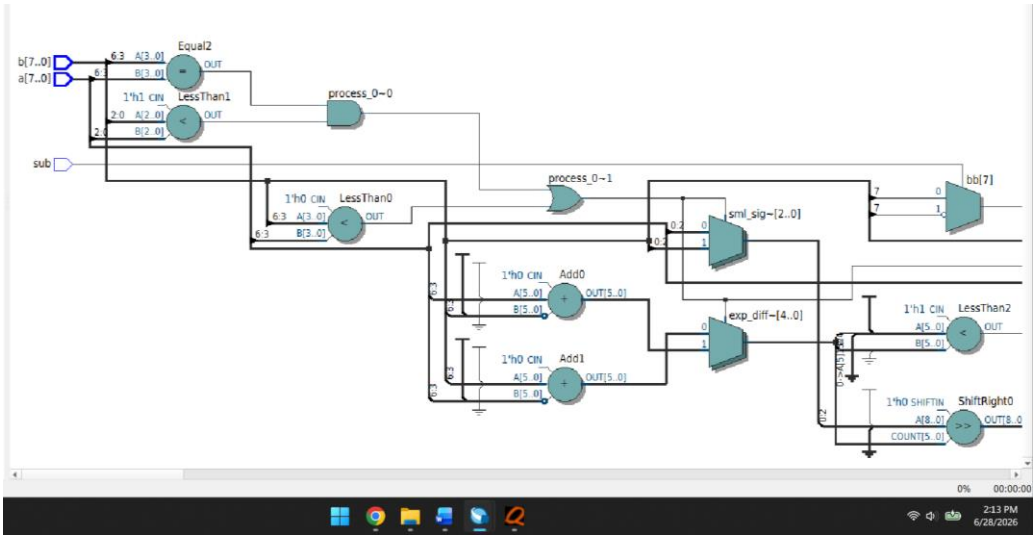


Figure 15 : vue RTL de lentrer de l'additionneur/soustracteur

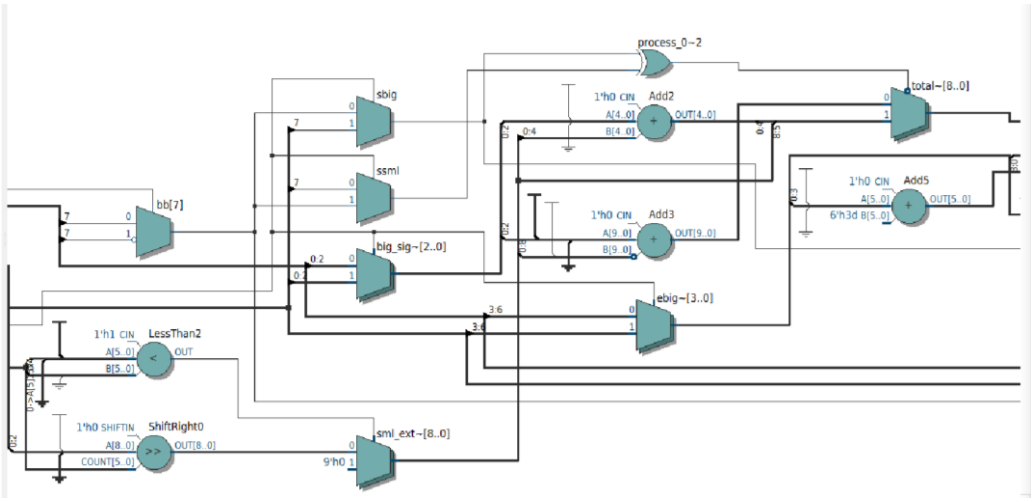


Figure 16 : vue RTL plus detailler de l'additionneur/soustracteur

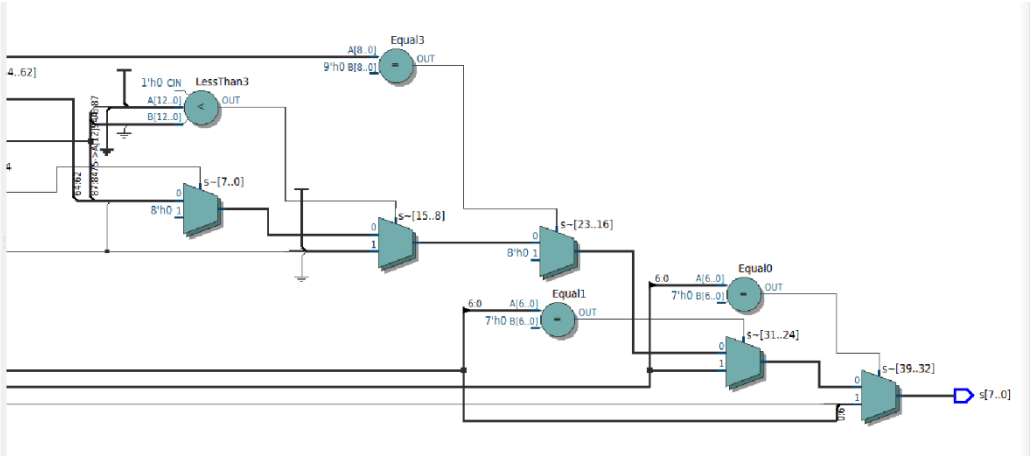
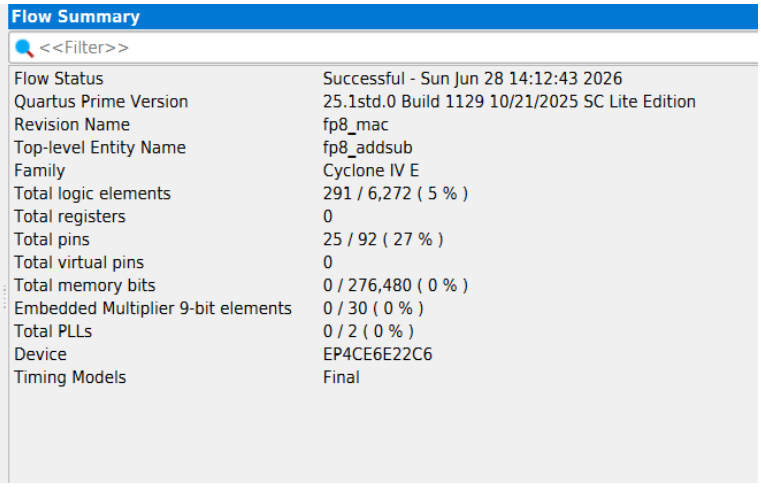


Figure 17 : vue RTL de sortie de l'additionneur/soustracteur

Le Flow Summary confirme que ce module est le plus gourmand. Il consomme 291 éléments logiques sur les 6 272 disponibles, soit environ 5 % du composant. C'est près de cinq fois plus que le multiplieur (66 éléments) et plus de cent fois plus que le décodeur (2 éléments). Ce contraste est très instructif : il montre que, contrairement à l'intuition, c'est l'addition et non la multiplication qui coûte le plus cher en virgule flottante. La raison est précisément l'alignement et la renormalisation, qui demandent des décaleurs et beaucoup de logique de contrôle, là où la multiplication se contente d'un petit multiplieur et d'une addition d'exposants. Comme les autres modules combinatoires, l'additionneur n'utilise aucun registre. Ce résultat justifie a posteriori un choix fait au chapitre 4 : commencer par le multiplieur, plus simple, avant d'aborder l'additionneur, qui concentre la vraie difficulté.


Flow Summary

<<Filter>>

Flow Status	Successful - Sun Jun 28 14:12:43 2026
Quartus Prime Version	25.1std.0 Build 1129 10/21/2025 SC Lite Edition
Revision Name	fp8_mac
Top-level Entity Name	fp8_addsub
Family	Cyclone IV E
Total logic elements	291 / 6,272 (5 %)
Total registers	0
Total pins	25 / 92 (27 %)
Total virtual pins	0
Total memory bits	0 / 276,480 (0 %)
Embedded Multiplier 9-bit elements	0 / 30 (0 %)
Total PLLs	0 / 2 (0 %)
Device	EP4CE6E22C6
Timing Models	Final

Figure 18 : Flow Summary de l'additionneur/soustracteur

5.2.4 L'accumulateur (fp8_acc)

L'accumulateur se distingue fondamentalement des trois modules précédents : c'est le seul qui est séquentiel. Là où le décodeur, le multiplieur et l'additionneur calculent une sortie qui suit instantanément leurs entrées, l'accumulateur, lui, doit mémoriser une valeur d'un cycle d'horloge au suivant. C'est ce qui permet au MAC d'additionner les produits les uns après les autres : sans cette mémoire, chaque calcul effacerait le précédent. Le module reçoit donc une horloge clk, un reset asynchrone, un effacement synchrone clear, une autorisation enable, une donnée d'entrée d_in, et fournit la valeur mémorisée sur q_out.

La vue RTL traduit visuellement cette nature séquentielle. L'élément central du schéma est un registre 8 bits, repéré reg[7..0], reconnaissable à ses entrées caractéristiques : une entrée de données D, une entrée d'horloge CLK, une entrée d'effacement SCLR (Synchronous Clear) et une entrée CLRN reliée au reset. C'est ce registre qui constitue la mémoire du module. En amont, on observe deux multiplexeurs en cascade (les blocs reg~[7..0] et reg~[15..8]) : ils déterminent ce qui sera chargé dans le registre au prochain front d'horloge. Le premier choisit, selon le signal enable, entre conserver la valeur actuelle ou charger la nouvelle donnée d_in ; le second gère le cas de l'effacement, en forçant la valeur à zéro lorsque clear est actif. Cette logique de sélection en entrée du registre est la traduction directe de notre description VHDL, où le comportement dépendait de

l'ordre de priorité entre reset, clear et enable. Le contraste avec les schémas précédents est frappant : ici, le circuit est petit, mais il contient un véritable élément de mémorisation, ce que les modules combinatoires n'avaient pas.

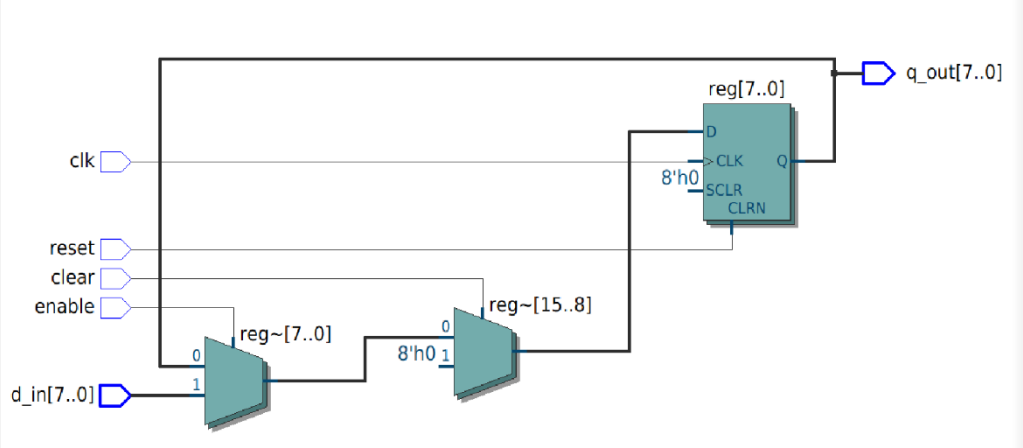


Figure 19 : : vue RTL de l'accumulateur

Le Flow Summary confirme cette particularité de façon très claire. L'accumulateur n'utilise que 9 éléments logiques, ce qui est très peu, mais surtout il mobilise 8 registres, alors que tous les modules précédents en utilisaient zéro. Ces 8 registres correspondent exactement aux 8 bits mémorisés par l'accumulateur. C'est la signature matérielle d'un circuit séquentiel : la présence de bascules. Ce module illustre donc concrètement la différence entre logique combinatoire et logique séquentielle, une distinction fondamentale en conception numérique. Le faible nombre d'éléments logiques s'explique par le fait qu'un registre, en lui-même, ne fait pas de calcul : il se contente de retenir une valeur, et seule la petite logique de sélection en entrée consomme quelques portes.

Flow Summary

<<Filter>>

Flow Status	Successful - Sun Jun 28 14:13:52 2026
Quartus Prime Version	25.1std.0 Build 1129 10/21/2025 SC Lite Edition
Revision Name	fp8_mac
Top-level Entity Name	fp8_acc
Family	Cyclone IV E
Total logic elements	9 / 6,272 (< 1 %)
Total registers	8
Total pins	20 / 92 (22 %)
Total virtual pins	0
Total memory bits	0 / 276,480 (0 %)
Embedded Multiplier 9-bit elements	0 / 30 (0 %)
Total PLLs	0 / 2 (0 %)
Device	EP4CE6E22C6
Timing Models	Final

Figure 20 : Flow Summary de l'accumulateur

5.2.5 Le MAC complet (fp8_mac)

Le module fp8_mac est le sommet de l'architecture : c'est lui qui relie tous les autres pour former l'opérateur complet. Il ne contient lui-même presque aucune logique de calcul, son rôle est essentiellement d'assembler et de connecter les sous-modules. C'est pour cela que sa vue RTL est, paradoxalement, l'une des plus lisibles : au lieu de portes élémentaires, on y voit directement les blocs fonctionnels que nous avons décrits un à un.

Le schéma se lit comme le chemin de la donnée à travers l'opérateur, et il correspond exactement à l'architecture proposée au chapitre 4 (figure 5). À gauche, les deux entrées a et b arrivent dans le bloc fp8_mult:u_mult, le multiplieur, qui calcule leur produit p. Ce produit est ensuite envoyé vers le bloc fp8_addsub:u_addsub, l'additionneur/soustracteur, dont la seconde entrée vient de la valeur déjà accumulée. Sa sortie s est dirigée vers le bloc fp8_acc:u_acc, l'accumulateur, qui la mémorise au rythme de l'horloge et la renvoie à la fois vers la sortie result et en retour vers l'additionneur, formant ainsi la boucle d'accumulation. On distingue aussi, en haut, un petit registre isolé nommé valid_r : c'est lui qui génère le signal valid, en retardant simplement le signal enable d'un cycle pour indiquer qu'une accumulation vient d'avoir lieu. La présence de ces trois grands blocs nommés (u_mult, u_addsub, u_acc), reliés exactement comme dans notre schéma de conception, prouve que l'outil a bien instancié et connecté nos modules comme prévu. C'est la confirmation que l'intention exprimée au chapitre 4 et la réalisation effective coïncident.

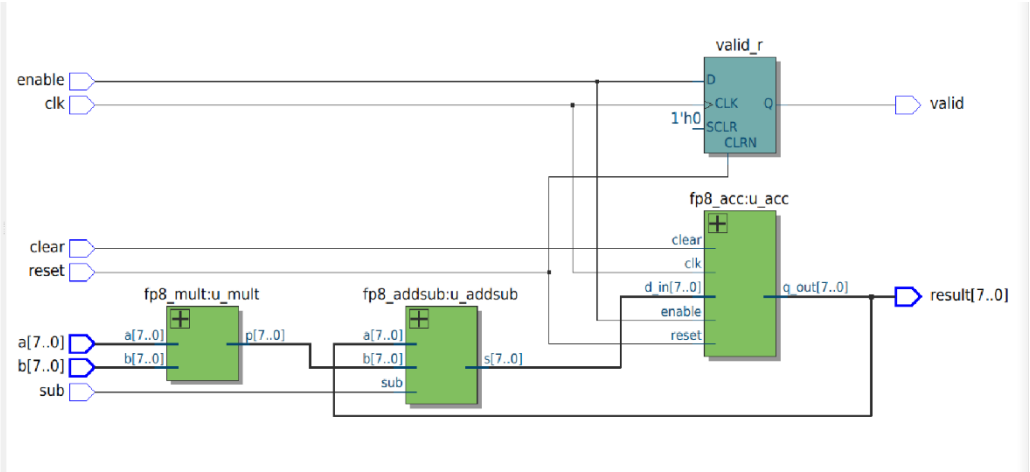
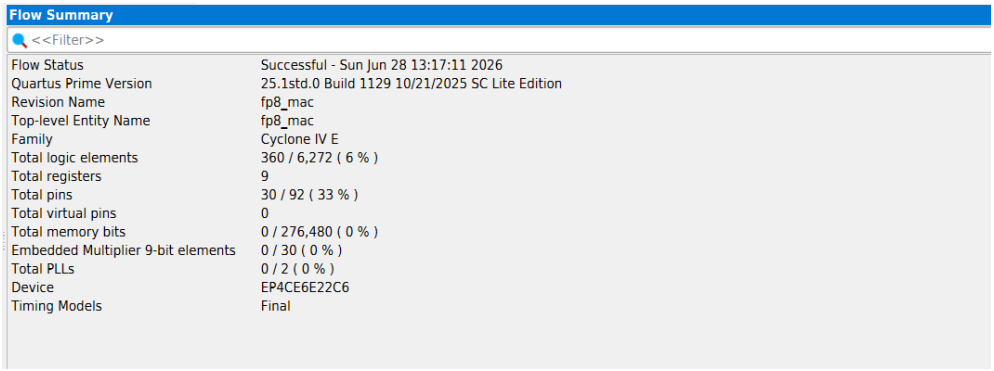


Figure 21 : vue RTL du MAC complet, à comparer avec la figure 5 du chapitre 4

Le Flow Summary du MAC complet donne enfin le bilan matériel de tout l'opérateur. L'ensemble occupe 360 éléments logiques sur les 6 272 disponibles, soit environ 6 % du composant, et 9 registres. On peut vérifier que ce total est cohérent avec la somme des modules pris séparément : le décodeur (2), le multiplieur (66), l'additionneur (291) et l'accumulateur (9) donnent un ordre de grandeur proche, l'additionneur représentant à lui seul la plus grosse part. Les 9 registres correspondent aux 8 bits de l'accumulateur plus le bit du signal valid. Le point le plus marquant reste la faiblesse globale de cette consommation : un opérateur multiplie-accumule complet, capable d'enchaîner des calculs, tient dans 6 % d'un petit FPGA d'entrée de gamme. C'est l'illustration concrète du message central de tout le projet : en travaillant sur 8 bits, on obtient des opérateurs si compacts qu'on pourrait en placer un très grand nombre en parallèle sur une même puce. C'est précisément ce qui fait l'intérêt du FP8 pour les accélérateurs d'intelligence artificielle.



Flow Summary

<<Filter>>

Flow Status	Successful - Sun Jun 28 13:17:11 2026
Quartus Prime Version	25.1std.0 Build 1129 10/21/2025 SC Lite Edition
Revision Name	fp8_mac
Top-level Entity Name	fp8_mac
Family	Cyclone IV E
Total logic elements	360 / 6,272 (6 %)
Total registers	9
Total pins	30 / 92 (33 %)
Total virtual pins	0
Total memory bits	0 / 276,480 (0 %)
Embedded Multiplier 9-bit elements	0 / 30 (0 %)
Total PLLs	0 / 2 (0 %)
Device	EP4CE6E22C6
Timing Models	Final

Figure 22 : : Flow Summary du MAC complet

Au terme de cette section, chaque module a été décrit, vérifié visuellement par son schéma RTL et chiffré par ses ressources. L'ensemble forme une architecture cohérente, dont on a confirmé qu'elle se traduit bien en matériel réel. Il reste à présent à vérifier non plus la structure, mais le comportement : c'est l'objet des bancs de test et des simulations.

5.3 Les bancs de test

Une fois la structure des modules confirmée, il restait à vérifier leur comportement, c'est-à-dire à s'assurer qu'ils calculent juste. Pour cela, nous avons écrit un banc de test (testbench) par module. Le principe que nous avons suivi est simple mais rigoureux : pour chaque cas, on applique en entrée des valeurs FP8 dont on connaît à l'avance le résultat, calculé à la main, puis on compare ce résultat attendu avec ce que produit réellement le module en simulation. Si les deux coïncident, le module est validé pour ce cas.

Le choix des cas de test n'a rien d'arbitraire. Pour chaque module, nous avons volontairement mélangé des cas « ordinaires » et des cas « limites », car ce sont surtout les cas limites qui révèlent les éventuels défauts. On y trouve ainsi des valeurs entières simples, des valeurs avec virgule, des nombres négatifs, mais aussi les situations particulières que sont le zéro, le dépassement de capacité (saturation), le soupassement et l'annulation. Cette variété permet de couvrir l'ensemble du fonctionnement de chaque opérateur, et pas seulement le cas le plus favorable.

Toute la simulation a été réalisée sous Questa, en mode autonome : les modules et leurs bancs de test sont compilés, puis la simulation est lancée et les signaux sont observés dans une fenêtre de chronogrammes. Pour faciliter la lecture, les valeurs sont affichées en binaire, ce qui permet de retrouver directement les trois champs du format FP8.

5.4 Résultats de simulation

Nous présentons ici les résultats obtenus pour chaque module, dans l'ordre où ils ont été développés. Pour chacun, un tableau résume les cas testés et leur résultat, et le chronogramme correspondant en apporte la preuve visuelle.

5.4.1 Le décodeur

Le décodeur a été testé sur dix valeurs couvrant tout l'éventail du format : des positifs, des négatifs, des valeurs avec mantisse, la plus grande valeur représentable, la plus petite valeur normale, et le zéro. Les résultats sont rassemblés dans le tableau suivant.

Tableau 4 : Résultats du décodeur

Entrée (fp8_in)	sign	exponent	mantissa	is_zero	Valeur
00111000	0	0111	000	0	+1,0
00111100	0	0111	100	0	+1,5
00111010	0	0111	010	0	+1,25
01001100	0	1001	100	0	+6,0
01011000	0	1011	000	0	+16,0
10111000	1	0111	000	0	−1,0
11001100	1	1001	100	0	−6,0
01111110	0	1111	110	0	+448
00001000	0	0001	000	0	+0,015625
00000000	0	0000	000	1	0

Le chronogramme confirme que, pour chaque entrée, les trois champs sont correctement extraits. On vérifie par exemple que l'entrée 00111000 donne bien un signe nul, un exposant 0111 et une mantisse 000, soit la valeur 1,0. On observe aussi que le signal is_zero reste à 0 pour toutes les valeurs non nulles et ne passe à 1 que pour la dernière entrée, 00000000. La détection du zéro fonctionne donc correctement, tout comme la gestion du signe, visible sur les entrées négatives 10111000 et 11001100. Le décodeur est ainsi validé sur toute la plage du format, de la plus petite à la plus grande valeur.



Figure 23 : chronogramme du décodeur HEXA

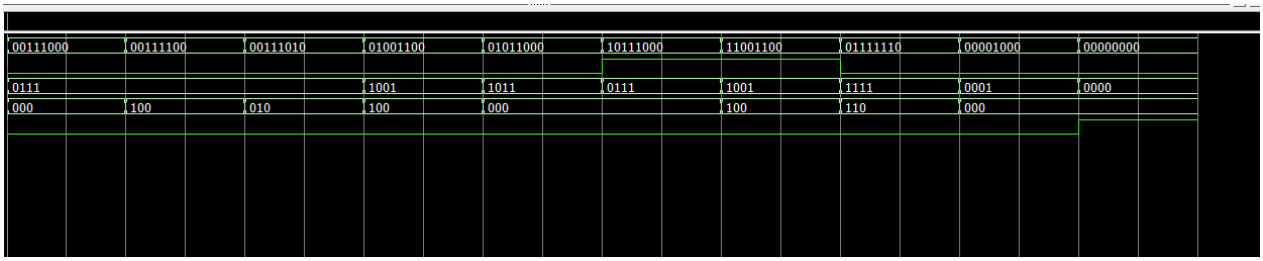


Figure 24 : chronogramme du décodeur binaire

5.4.2 Le multiplieur

Le multiplieur a été testé sur quinze cas, choisis pour couvrir aussi bien des produits exacts que des situations problématiques. Le tableau ci-dessous donne, pour chaque opération, le résultat attendu et le résultat obtenu en simulation.

Tableau 5 : Résultats du multiplieur

Opération	a	b	p (obtenu)	Valeur	Remarque
1,5 × 1,5	00111100	00111100	01000001	2,25	exact
1,5 × 2,5	00111100	01000010	01000111	3,75	exact
0,75 × 0,75	00110100	00110100	00110001	0,5625	exact
1,25 × 2,0	00111010	01000000	01000010	2,5	exact
1,25 × 1,25	00111010	00111010	00111100	1,5	arrondi (réel 1,5625)
2,5 × 2,5	01000010	01000010	01001100	6,0	arrondi (réel 6,25)
1,0 × 6,0	00111000	01001100	01001100	6,0	exact
2,0 × 2,0	01000000	01000000	01001000	4,0	exact

6,0 × 6,0	01001100	01001100	01100001	36,0	normalisation
16 × 16	01011000	01011000	01111000	256	grand nombre
32 × 32	01100000	01100000	01111110	448	saturation
−1,0 × 6,0	10111000	01001100	11001100	−6,0	signe négatif
−2,5 × −2,0	11000010	11000000	01001010	+5,0	deux négatifs
très petit²	00001000	00001000	00000000	~0	soupassement
0 × 6,0	00000000	01001100	00000000	0	zéro

Le chronogramme confirme l'ensemble de ces résultats, et plusieurs cas méritent qu'on s'y attarde. Le produit 32 × 32 vaudrait normalement 1024, soit bien au-delà de la limite du format ; on observe que le résultat est ramené à 448 (01111110), ce qui valide notre mécanisme de saturation. Le cas −2,5 × −2,0 donne un résultat positif (+5,0), ce qui confirme que le signe est correctement calculé par le XOR des deux signes. Le cas du produit de deux très petites valeurs donne zéro, illustrant la gestion du soupassement. Enfin, les cas comme 1,25 × 1,25 sont particulièrement instructifs : le résultat exact serait 1,5625, mais comme cette valeur n'est pas représentable en FP8, le circuit tronque la mantisse et renvoie la valeur codable immédiatement inférieure, 1,5.

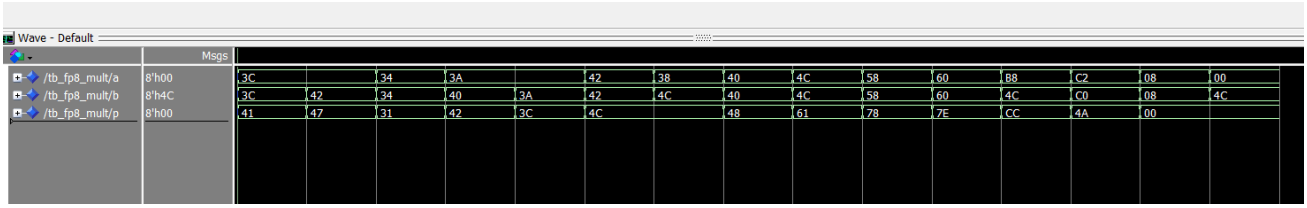


Figure 25 : : chronogramme du multiplieur HEXA

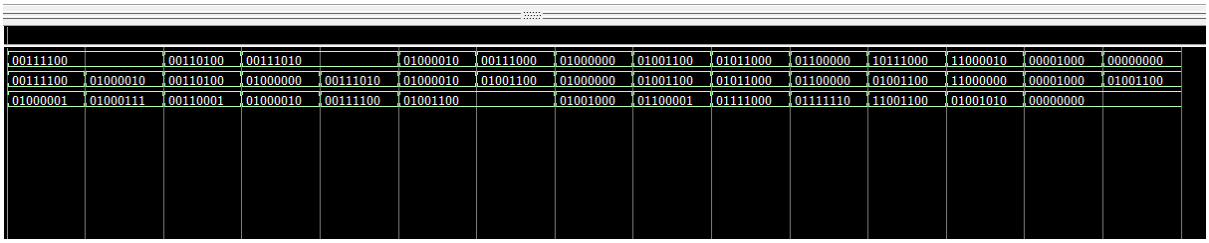


Figure 26 : chronogramme du multiplieur binaire

5.4.3 L'additionneur/soustracteur

L'additionneur/soustracteur a été testé sur douze cas, incluant des additions, des soustractions (signal sub à 1), des cas d'alignement d'exposants, une annulation, un résultat négatif et une saturation. Le tableau suivant récapitule les résultats.

Tableau 6 : Résultats de l'additionneur/soustracteur

Opération	sub	s (obtenu)	Valeur	Remarque
1,0 + 1,0	0	01000000	2,0	simple
2,0 + 1,0	0	01000100	3,0	simple
6,0 + 1,0	0	01001110	7,0	alignement
8,0 + 1,0	0	01010001	9,0	alignement
1,5 + 1,5	0	01000100	3,0	avec virgule
1,25 + 1,0	0	01000001	2,25	avec virgule
6,0 − 2,0	1	01001000	4,0	soustraction
6,0 − 4,0	1	01000000	2,0	normalisation
6,0 − 6,0	1	00000000	0	annulation
1,0 − 6,0	1	11001000	−5,0	résultat négatif
448 + 448	0	01111110	448	saturation
6,0 + 0	0	01001100	6,0	avec zéro

Le chronogramme confirme ces résultats et met en évidence le bon fonctionnement des étapes les plus délicates du module. Les cas 6,0 + 1,0 et 8,0 + 1,0 montrent que l'alignement des exposants fonctionne : les deux nombres, qui n'ont pas le même ordre de grandeur, sont correctement mis à la même échelle avant d'être additionnés. Le cas 6,0 − 6,0 donne exactement zéro, ce qui valide la gestion de l'annulation, un cas particulièrement sensible puisqu'il demande de détecter que le résultat est nul. Le cas 1,0 − 6,0 produit un résultat négatif (−5,0, soit 11001000 avec le bit de signe à 1), ce qui confirme que le signe du résultat est bien déterminé d'après le plus grand des deux opérandes. Enfin, l'addition 448 + 448, dont le résultat mathématique dépasse la capacité du format, est correctement ramenée à 448 par la saturation. Le signal sub joue bien son rôle : à 1, il transforme l'addition en soustraction, comme le montrent les cas 6,0 − 2,0 et 6,0 − 4,0.

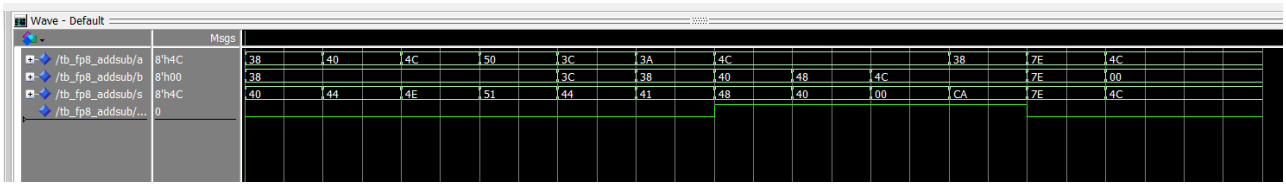


Figure 27 : chronogramme de l'additionneur/soustracteur HEXA

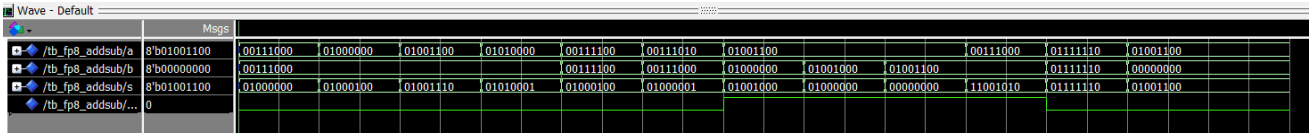


Figure 28 : chronogramme de l'additionneur/soustracteur binaire

5.4.4 Le MAC complet

Le MAC est le test le plus complet, puisqu'il met en œuvre tous les modules à la fois, dans un fonctionnement séquentiel cadencé par l'horloge. Le banc de test enchaîne plusieurs accumulations, puis une remise à zéro par clear, une nouvelle accumulation et enfin une soustraction, afin de vérifier l'ensemble des fonctionnalités. L'évolution de la sortie result est résumée ci-dessous.

Tableau 7 : Évolution de l'accumulateur du MAC

Étape	Opération	result	Valeur
reset	—	00000000	0
accumulation	+ (1,0 × 1,0)	00111000	1,0
accumulation	+ (2,0 × 1,0)	01000100	3,0
accumulation	+ (6,0 × 1,0)	01010001	9,0
clear	remise à zéro	00000000	0
accumulation	+ (2,0 × 2,0)	01001000	4,0
accumulation	+ (1,5 × 2,0)	01001110	7,0
soustraction	− (2,0 × 1,0)	01001010	5,0

Le chronogramme du MAC est la preuve la plus marquante du bon fonctionnement de l'opérateur complet. On y suit la valeur de result qui s'élève progressivement, cycle après cycle : après le reset, elle vaut successivement 1,0, puis 3,0, puis 9,0, chaque nouvelle valeur correspondant bien à l'ancienne plus le nouveau produit. Lorsque le signal clear est activé, result retombe à zéro, ce qui permet de démarrer une nouvelle série de calculs : on obtient alors 4,0 puis 7,0. La dernière étape utilise le signal sub pour retrancher un produit, et la valeur redescend à 5,0. On vérifie également

que le signal valid accompagne les cycles d'accumulation actifs. Toutes les transitions se produisent sur les fronts montants de l'horloge, ce qui confirme le caractère synchrone du circuit. Comme le MAC utilise simultanément le multiplieur, l'additionneur et l'accumulateur, ce chronogramme prouve non seulement que chaque bloc fonctionne, mais surtout qu'ils fonctionnent correctement ensemble, ce qui était l'objectif final du projet.

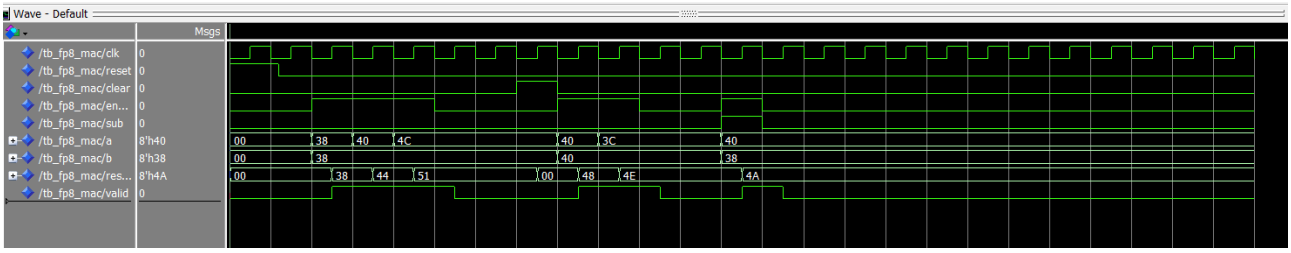


Figure 29 : chronogramme du MAC complet HEXA

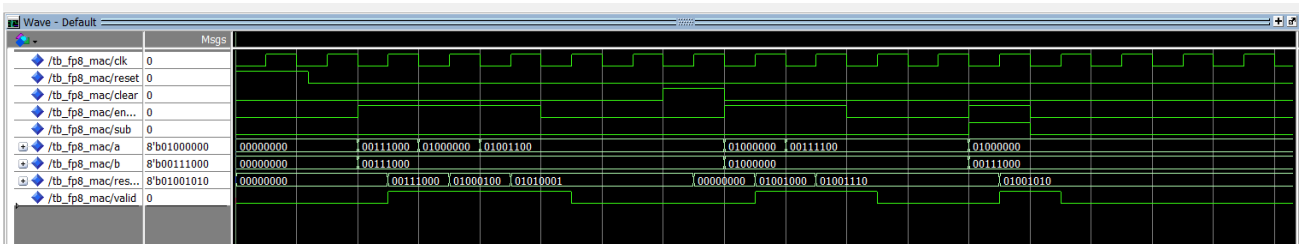


Figure 30 : chronogramme du MAC complet binaire

5.5 Analyse des résultats, ressources et limites

Les simulations présentées plus haut confirment que l'opérateur fonctionne correctement, aussi bien sur les cas ordinaires que sur les cas limites. Il reste maintenant à prendre du recul sur ces résultats : que nous apprennent-ils sur le coût matériel de notre architecture, et sur les limites du format que nous avons choisi ?

Bilan des ressources.

En mettant successivement chaque module en entité de plus haut niveau, nous avons pu relever ses ressources propres dans le rapport de synthèse. Le tableau suivant rassemble ces chiffres.

Tableau 8 : Ressources logiques utilisées par chaque module (composant Cyclone IV E, 6 272 éléments logiques disponibles)

Module	Éléments logiques	Registres	Type
fp8_decoder	2	0	combinatoire
fp8_mult	66	0	combinatoire
fp8_addsub	291	0	combinatoire
fp8_acc	9	8	séquentiel
fp8_mac (complet)	**360**	**9**	**séquentiel**

Plusieurs enseignements ressortent de ce tableau. Le premier, et le plus parlant, est la faiblesse globale de la consommation : l'opérateur complet n'occupe que 360 éléments logiques, soit environ 6 % d'un petit FPGA d'entrée de gamme. Autrement dit, on pourrait théoriquement placer une quinzaine de ces opérateurs sur une seule puce de ce type. C'est exactement l'argument avancé au chapitre 1 en faveur du FP8 : des opérateurs si compacts qu'on peut les multiplier pour paralléliser les calculs, ce qui est précisément ce que font les accélérateurs d'intelligence artificielle.

Le deuxième enseignement est la répartition très inégale de ce coût entre les modules. Le décodeur est presque gratuit (2 éléments logiques), parce qu'il ne fait que du câblage. Le multiplieur reste modeste (66 éléments). En revanche, l'additionneur consomme à lui seul 291 éléments, soit la plus grande partie du total. Ce résultat, qui peut surprendre au premier abord, confirme ce que nous avions annoncé au chapitre 4 : en virgule flottante, c'est l'addition, et non la multiplication, qui est l'opération la plus coûteuse, à cause de l'alignement des exposants et de la renormalisation. La présence des décaleurs et de toute la logique de contrôle associée explique cet écart. On notera aussi que Quartus n'a utilisé aucun multiplieur matériel dédié : la multiplication des mantisses, sur seulement 4 bits, est trop petite pour cela et se fait en logique ordinaire.

Enfin, le tableau met en évidence la frontière entre logique combinatoire et logique séquentielle. Seuls l'accumulateur et, par conséquent, le MAC complet utilisent des registres : 8 pour mémoriser la valeur accumulée, plus 1 pour le signal valid. Les modules de calcul, eux, n'en utilisent aucun, ce qui est cohérent avec leur nature purement combinatoire.

Limites de précision :

Si l'opérateur calcule juste, il faut être honnête sur le fait qu'il ne calcule pas toujours exactement. Plusieurs résultats de simulation l'ont montré, en particulier dans le multiplieur. Le cas le plus clair est celui de 1,25 × 1,25 : le résultat mathématique exact est 1,5625, mais le circuit renvoie 1,5. De même, 2,5 × 2,5 devrait donner 6,25, et l'on obtient 6,0. Ces écarts ne sont pas des erreurs de conception : ils viennent du format lui-même. Avec seulement 3 bits de mantisse, le FP8 ne dispose que de 256 valeurs représentables, et la plupart des nombres réels n'en font pas partie. Lorsqu'un résultat tombe entre deux valeurs codables, le circuit, qui arrondit par troncature, retient la valeur représentable immédiatement inférieure en valeur absolue (arrondi vers zéro), et non forcément la plus proche.

Ce phénomène s'aggrave à mesure que les nombres grandissent, car l'écart entre deux valeurs codables augmente avec leur ordre de grandeur. Autour de 1, le pas est de 0,125 ; autour de 256, il atteint 32. Un nombre comme 25,2 ne peut donc tout simplement pas être représenté : il se trouve entre 24 et 26. Un arrondi au plus proche donnerait 26, mais avec la troncature le circuit renvoie 24. Cette perte de précision est le prix à payer pour la compacité du format, et c'est un compromis assumé : le FP8 n'a jamais eu vocation à être précis, mais à être petit et rapide.

Limites liées à nos choix de conception :

À ces limites propres au format s'ajoutent celles que nous avons volontairement introduites pour garder un projet réalisable. Nous avons choisi un arrondi par troncature plutôt qu'un arrondi au plus proche, ce qui simplifie le matériel mais dégrade légèrement la précision. Nous n'avons pas implémenté les nombres sous-normaux, si bien que les valeurs très proches de zéro sont ramenées à zéro. Nous avons remplacé l'infini par une saturation à la valeur maximale. Enfin, et c'est sans doute la limite la plus importante, l'accumulation se fait entièrement en FP8 : à chaque cycle, le résultat est ramené sur 8 bits. Or, lorsqu'on additionne un grand nombre et un très petit, ce dernier peut être complètement « avalé » et disparaître du résultat. Dans un accélérateur réel, on accumule justement sur un format plus large pour éviter ce problème.

Ces limites ne remettent pas en cause la validité du projet. Elles correspondent à des décisions de conception clairement annoncées, prises pour qu'un étudiant puisse concevoir, coder, comprendre et vérifier seul l'ensemble de l'opérateur. Elles tracent surtout la voie vers des améliorations possibles, que nous évoquons en conclusion.

Conclusion générale et perspectives

Ce projet est parti d'un constat simple : les modèles d'intelligence artificielle sont devenus tellement gros que la manière même de représenter les nombres dans le matériel est devenue un enjeu. Le format FP8, sur seulement 8 bits, est l'une des réponses à ce problème, et c'est lui que nous avons cherché à comprendre puis à implémenter.

Pour cela, nous avons d'abord posé le contexte et rappelé les bases des nombres à virgule flottante, avant d'étudier le format FP8 en détail, avec ses deux variantes et ses compromis. Cette partie théorique nous a permis de faire un choix éclairé pour la conception : la variante E4M3, et un opérateur de type multiplieur-accumulateur (MAC), car c'est l'opération que l'on retrouve réellement au cœur des réseaux de neurones.

Nous avons ensuite conçu cette architecture bloc par bloc, puis nous l'avons réalisée en VHDL sous Quartus : un décodeur, un multiplieur, un additionneur/soustracteur, un accumulateur synchrone, et un module de plus haut niveau reliant le tout. Chaque module a été vérifié de deux façons complémentaires. D'un côté, la synthèse sous Quartus a fourni le schéma RTL de chaque bloc, confirmant que notre code décrit bien un circuit réel, ainsi que le coût en ressources : l'opérateur complet tient dans environ 6 % d'un petit FPGA, et l'on a pu vérifier concrètement que l'additionneur est le module le plus gourmand, alors que la multiplication, sur si peu de bits, reste très économe. De l'autre, la simulation sous Questa a confirmé, chronogramme à l'appui, que l'opérateur calcule correctement, aussi bien sur les cas simples que sur les cas particuliers comme le zéro, la saturation, l'annulation ou la remise à zéro de l'accumulateur.

Au-delà du résultat technique, ce projet nous a surtout fait toucher du doigt le fonctionnement interne du calcul flottant. On comprend mieux, après l'avoir câblé soi-même, pourquoi la multiplication est plus simple que l'addition, à quoi sert le bit caché, ou pourquoi la normalisation est une étape incontournable. Le fait de travailler sur seulement 8 bits a rendu ces mécanismes plus lisibles que sur un format classique, et le passage par le RTL Viewer et le rapport de ressources a permis de relier directement le code écrit au matériel qu'il engendre.

Bien sûr, notre réalisation reste volontairement simplifiée. La précision est limitée par les 3 bits de mantisse et par l'arrondi par troncature, comme l'ont clairement montré certains résultats de simulation où un produit exact n'était pas représentable et se trouvait tronqué à la valeur codable immédiatement inférieure. L'accumulation se fait directement en FP8, ce qui fait disparaître les petites
contributions, et plusieurs cas définis par la norme, comme les sous-normaux ou l'infini, ne sont pas gérés. Ces limites sont assumées : elles correspondent à des choix faits pour garder un projet réalisable seul, mais elles ouvrent naturellement la voie à des améliorations.

Perspectives :

Plusieurs prolongements sont envisageables. Le plus immédiat serait de remplacer la troncature par un véritable arrondi au plus proche, afin de gagner en précision. On pourrait aussi accumuler en précision étendue (par exemple sur 16 bits) plutôt qu'en FP8, comme le font les accélérateurs réels, pour éviter la perte des petites valeurs. La gestion des sous-normaux et des cas spéciaux (infini, NaN) rapprocherait l'opérateur de la norme. À plus grande échelle, on pourrait mettre plusieurs MAC en parallèle pour calculer un produit de matrices, ce qui constitue la brique de base d'un véritable accélérateur d'IA. Enfin, il serait intéressant de pipeliner l'architecture pour augmenter la fréquence de fonctionnement, et pourquoi pas, à terme, de l'implanter réellement sur une carte FPGA, l'étape de synthèse ayant déjà confirmé que le circuit est parfaitement synthétisable. Autant de pistes qui montrent que ce projet, malgré sa simplicité, constitue une première marche solide vers des architectures bien plus ambitieuses.

Bibliographie

[1] P. Micikevicius, D. Stosic, N. Burgess, et al., « FP8 Formats for Deep Learning », arXiv preprint arXiv:2209.05433, 2022.

[2] Open Compute Project, « OCP 8-bit Floating Point Specification (OFP8) », Revision 1.0, 2023.

[3] IEEE, « IEEE Standard for Floating-Point Arithmetic », IEEE Std 754-2019, 2019.

[4] D. Goldberg, « What Every Computer Scientist Should Know About Floating-Point Arithmetic », ACM Computing Surveys, vol. 23, n° 1, p. 5-48, 1991.

[5] B. Parhami, Computer Arithmetic: Algorithms and Hardware Designs, 2e éd., Oxford University Press, 2010.

[6] P. J. Ashenden, The Designer's Guide to VHDL, 3e éd., Morgan Kaufmann, 2008.

[7] NVIDIA Corporation, « NVIDIA H100 Tensor Core GPU Architecture » (Hopper Architecture Whitepaper), 2022.

[8] Intel Corporation, Intel Quartus Prime Standard Edition User Guide et Questa Intel FPGA Edition Simulation User Guide, 2024-2025.

Annexes :

fp8_pkg :

```
library ieee;
use ieee.std_logic_1164.all;

--  Package : parametres du format FP8  (variante E4M3)
package fp8_pkg is
    constant FP8_WIDTH : integer := 8;
    constant EXP_WIDTH : integer := 4;
    constant MAN_WIDTH : integer := 3;
    constant EXP_BIAS  : integer := 7;    -- biais de l'expoosant

    subtype fp8_t is std_logic_vector(FP8_WIDTH-1 downto 0);
end package fp8_pkg;
```

fp8_decoder

```
library ieee;
use ieee.std_logic_1164.all;
use work.fp8_pkg.all;

-- Decodeur FP8 : separe un mot de 8 bits en signe / exposant / mantisse
entity fp8_decoder is
    port (
        fp8_in   : in  std_logic_vector(7 downto 0);
        sign     : out std_logic;
        exponent : out std_logic_vector(3 downto 0);
        mantissa : out std_logic_vector(2 downto 0);
        is_zero  : out std_logic
    );
end entity fp8_decoder;

architecture rtl of fp8_decoder is
begin
    sign     <= fp8_in(7);
    exponent <= fp8_in(6 downto 3);
    mantissa <= fp8_in(2 downto 0);
    is_zero  <= '1' when (fp8_in(6 downto 0) = "0000000") else '0';
end architecture rtl;
```

fp8_acc :

```
library ieee;
use ieee.std_logic_1164.all;

-- Registre accumulateur synchrone 8 bits
entity fp8_acc is
    port (
        clk    : in  std_logic;
        reset  : in  std_logic;
        clear  : in  std_logic;
        enable : in  std_logic;
        d_in   : in  std_logic_vector(7 downto 0);
        q_out  : out std_logic_vector(7 downto 0)
```

```
    );
end entity fp8_acc;

architecture rtl of fp8_acc is
    signal reg : std_logic_vector(7 downto 0) := (others => '0');
begin
    process(clk, reset)
    begin
        if reset = '1' then
            reg <= (others => '0');
        elsif rising_edge(clk) then
            if clear = '1' then
                reg <= (others => '0');
            elsif enable = '1' then
                reg <= d_in;
            end if;
        end if;
    end process;

    q_out <= reg;
end architecture rtl;
```

fp8_mult :

```
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;
use work.fp8_pkg.all;

-- Multiplieur FP8 (E4M3) combinatoire : p = a * b
entity fp8_mult is
    port (
        a : in  std_logic_vector(7 downto 0);
        b : in  std_logic_vector(7 downto 0);
        p : out std_logic_vector(7 downto 0)
    );
end entity fp8_mult;

architecture rtl of fp8_mult is
begin
    process(a, b)
        variable sa, sb, sp  : std_logic;
        variable ea, eb      : integer;
        variable sig_a, sig_b: unsigned(3 downto 0);
        variable prod        : unsigned(7 downto 0);
        variable exp_sum     : integer;
        variable exp_res     : integer;
        variable man_res     : std_logic_vector(2 downto 0);
        variable zero_a, zero_b : boolean;
    begin
        -- Extraction des champs
        sa := a(7);  sb := b(7);
        ea := to_integer(unsigned(a(6 downto 3)));
```

```
        eb := to_integer(unsigned(b(6 downto 3)));
        zero_a := (a(6 downto 0) = "0000000");
        zero_b := (b(6 downto 0) = "0000000");

        -- Signe du produit (XOR des signes)
        sp := sa xor sb;

        -- Significandes avec bit cache (1.xxx -> 4 bits)
        sig_a := unsigned('1' & a(2 downto 0));
        sig_b := unsigned('1' & b(2 downto 0));

        -- Multiplication des significandes (4x4 = 8 bits)
        prod := sig_a * sig_b;

        -- Addition des exposants moins le biais
        exp_sum := ea + eb - EXP_BIAS;

        -- Normalisation : le produit vaut entre 1 et presque 4
        if prod(7) = '1' then                -- produit >= 2.0
            man_res := std_logic_vector(prod(6 downto 4));
            exp_res := exp_sum + 1;
        else                                  -- produit dans [1,2)
            man_res := std_logic_vector(prod(5 downto 3));
            exp_res := exp_sum;
        end if;

        -- Cas particuliers + assemblage
        if zero_a or zero_b then
            p <= (others => '0');                                    --
resultat nul
        elsif exp_res > 15 then
            p <= sp & "1111" & "110";                                --
saturation a 448
        elsif exp_res < 1 then
            p <= (others => '0');                                    --
soupassement -> 0
        else
            p <= sp & std_logic_vector(to_unsigned(exp_res, 4)) &
man_res;
        end if;
    end process;
end architecture rtl;
```

fp8_addsub :

```
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;
use work.fp8_pkg.all;

-- Additionneur / soustracteur FP8 (E4M3)
--   sub = '0' : s = a + b
--   sub = '1' : s = a - b
entity fp8_addsub is
    port (
```

```
        a   : in  std_logic_vector(7 downto 0);
        b   : in  std_logic_vector(7 downto 0);
        sub : in  std_logic;
        s   : out std_logic_vector(7 downto 0)
    );
end entity fp8_addsub;

architecture rtl of fp8_addsub is
begin
    process(a, b, sub)
        variable bb         : std_logic_vector(7 downto 0);
        variable sa, sb     : std_logic;
        variable ea, eb     : integer;
        variable sbig, ssml: std_logic;
        variable ebig       : integer;
        variable big_sig, sml_sig : unsigned(3 downto 0);
        variable big_ext, sml_ext, total : unsigned(8 downto 0);
        variable exp_diff, exp_res : integer;
        variable man_res    : std_logic_vector(2 downto 0);
        variable a_big      : boolean;
    begin
        -- 1. Application du sub : on inverse le signe de b si on
soustrait
        bb := b;
        if sub = '1' then
            bb(7) := not b(7);
        end if;

        sa := a(7);  sb := bb(7);
        ea := to_integer(unsigned(a(6 downto 3)));
        eb := to_integer(unsigned(bb(6 downto 3)));

        -- 2. Cas des entrees nulles
        if a(6 downto 0) = "0000000" then
            s <= bb;
        elsif bb(6 downto 0) = "0000000" then
            s <= a;
        else
            -- 3. Determiner le plus grand en valeur absolue
            if (ea > eb) or
               (ea = eb and unsigned(a(2 downto 0)) >= unsigned(bb(2
downto 0))) then
                a_big := true;
            else
                a_big := false;
            end if;

            if a_big then
                sbig := sa; ssml := sb; ebig := ea;
                big_sig := unsigned('1' & a(2 downto 0));
                sml_sig := unsigned('1' & bb(2 downto 0));
                exp_diff := ea - eb;
            else
                sbig := sb; ssml := sa; ebig := eb;
                big_sig := unsigned('1' & bb(2 downto 0));
                sml_sig := unsigned('1' & a(2 downto 0));
                exp_diff := eb - ea;
            end if;
```

```
            -- 4. Extension avec bits de garde
            big_ext := "0" & big_sig & "0000";
            sml_ext := "0" & sml_sig & "0000";

            if exp_diff >= 9 then
                sml_ext := (others => '0');
            else
                sml_ext := shift_right(sml_ext, exp_diff);
            end if;

            -- 5. Addition ou soustraction selon les signes
            if sbig = ssml then
                total := big_ext + sml_ext;
            else
                total := big_ext - sml_ext;
            end if;

            -- 6. Normalisation
            exp_res := ebig;
            if total = 0 then
                s <= (others => '0');
            else
                if total(8) = '1' then
                    total := shift_right(total, 1);
                    exp_res := exp_res + 1;
                else
                    for i in 0 to 7 loop
                        if total(7) = '0' then
                            total := shift_left(total, 1);
                            exp_res := exp_res - 1;
                        end if;
                    end loop;
                end if;

                man_res := std_logic_vector(total(6 downto 4));

                if exp_res > 15 then
                    s <= sbig & "1111" & "110";
                elsif exp_res < 1 then
                    s <= (others => '0');
                else
                    s <= sbig & std_logic_vector(to_unsigned(exp_res, 4))
& man_res;
                end if;
            end if;
        end if;
    end process;
end architecture rtl;
```

fp8_mac :

```
library ieee;
use ieee.std_logic_1164.all;
use work.fp8_pkg.all;

-- Top-level : multiplieur-accumulateur FP8
-- A chaque front d'horloge actif (enable) : acc = acc +/- (a * b)
```

```
entity fp8_mac is
    port (
        clk    : in  std_logic;
        reset  : in  std_logic;
        clear  : in  std_logic;
        enable : in  std_logic;
        sub    : in  std_logic;
        a      : in  std_logic_vector(7 downto 0);
        b      : in  std_logic_vector(7 downto 0);
        result : out std_logic_vector(7 downto 0);
        valid  : out std_logic
    );
end entity fp8_mac;

architecture rtl of fp8_mac is
    signal prod    : std_logic_vector(7 downto 0);
    signal sum     : std_logic_vector(7 downto 0);
    signal acc_q   : std_logic_vector(7 downto 0);
    signal valid_r : std_logic := '0';
begin
    u_mult : entity work.fp8_mult
        port map (a => a, b => b, p => prod);

    u_addsub : entity work.fp8_addsub
        port map (a => acc_q, b => prod, sub => sub, s => sum);

    u_acc : entity work.fp8_acc
        port map (clk => clk, reset => reset, clear => clear,
                  enable => enable, d_in => sum, q_out => acc_q);

    process(clk, reset)
    begin
        if reset = '1' then
            valid_r <= '0';
        elsif rising_edge(clk) then
            valid_r <= enable;
        end if;
    end process;

    result <= acc_q;
    valid  <= valid_r;
end architecture rtl;
```

tb_fp8_decoder

```
library ieee;
use ieee.std_logic_1164.all;

-- Banc de test du decodeur FP8 (E4M3)
entity tb_fp8_decoder is
end entity tb_fp8_decoder;

architecture sim of tb_fp8_decoder is
    signal fp8_in   : std_logic_vector(7 downto 0) := (others => '0');
    signal sign     : std_logic;
    signal exponent : std_logic_vector(3 downto 0);
    signal mantissa : std_logic_vector(2 downto 0);
```

```
    signal is_zero  : std_logic;
begin
    uut : entity work.fp8_decoder
        port map (fp8_in => fp8_in, sign => sign, exponent => exponent,
                  mantissa => mantissa, is_zero => is_zero);
    process
    begin
        fp8_in <= "00111000"; wait for 20 ns;  -- +1,0
        fp8_in <= "00111100"; wait for 20 ns;  -- +1,5
        fp8_in <= "00111010"; wait for 20 ns;  -- +1,25
        fp8_in <= "01001100"; wait for 20 ns;  -- +6,0
        fp8_in <= "01011000"; wait for 20 ns;  -- +16,0
        fp8_in <= "10111000"; wait for 20 ns;  -- -1,0
        fp8_in <= "11001100"; wait for 20 ns;  -- -6,0
        fp8_in <= "01111110"; wait for 20 ns;  -- +448 (max)
        fp8_in <= "00001000"; wait for 20 ns;  -- +0,015625
        fp8_in <= "00000000"; wait for 20 ns;  -- 0
        wait;
    end process;
end architecture sim;
```

tb_fp8_mult :

```
library ieee;
use ieee.std_logic_1164.all;

-- Banc de test du multiplieur FP8 (E4M3)
entity tb_fp8_mult is
end entity tb_fp8_mult;

architecture sim of tb_fp8_mult is
    signal a, b, p : std_logic_vector(7 downto 0) := (others => '0');
begin
    uut : entity work.fp8_mult port map (a => a, b => b, p => p);
    process
    begin
        a <= "00111100"; b <= "00111100"; wait for 20 ns; -- 1,5  * 1,5
= 2,25
        a <= "00111100"; b <= "01000010"; wait for 20 ns; -- 1,5  * 2,5
= 3,75
        a <= "00110100"; b <= "00110100"; wait for 20 ns; -- 0,75 * 0,75
= 0,5625
        a <= "00111010"; b <= "01000000"; wait for 20 ns; -- 1,25 * 2,0
= 2,5
        a <= "00111010"; b <= "00111010"; wait for 20 ns; -- 1,25 * 1,25
-> 1,5 (arrondie)
        a <= "01000010"; b <= "01000010"; wait for 20 ns; -- 2,5  * 2,5
-> 6,0 (arrondi)
        a <= "00111000"; b <= "01001100"; wait for 20 ns; -- 1,0  * 6,0
= 6,0
        a <= "01000000"; b <= "01000000"; wait for 20 ns; -- 2,0  * 2,0
= 4,0
```

```
        a <= "01001100"; b <= "01001100"; wait for 20 ns; -- 6,0  * 6,0
= 36,0
        a <= "01011000"; b <= "01011000"; wait for 20 ns; -- 16   * 16
= 256
        a <= "01100000"; b <= "01100000"; wait for 20 ns; -- 32   * 32
-> 448 (saturation)
        a <= "10111000"; b <= "01001100"; wait for 20 ns; -- -1,0 * 6,0
= -6,0
        a <= "11000010"; b <= "11000000"; wait for 20 ns; -- -2,5 * -2,0
= +5,0
        a <= "00001000"; b <= "00001000"; wait for 20 ns; -- tres petit -
> ~0 (soupasssement)
        a <= "00000000"; b <= "01001100"; wait for 20 ns; -- 0    * 6,0
= 0
        wait;
    end process;
end architecture sim;
```

tb_fp8_addsub :

```
library ieee;
use ieee.std_logic_1164.all;

-- Banc de test de l'additionneur/sousstracteur FP8 (E4M3)
entity tb_fp8_addsub is
end entity tb_fp8_addsub;

architecture sim of tb_fp8_addsub is
    signal a, b, s : std_logic_vector(7 downto 0) := (others => '0');
    signal sub     : std_logic := '0';
begin
    uut : entity work.fp8_addsub port map (a => a, b => b, sub => sub, s
=> s);
    process
    begin
        a <= "00111000"; b <= "00111000"; sub <= '0'; wait for 20 ns; --
1,0 + 1,0 = 2,0
        a <= "01000000"; b <= "00111000"; sub <= '0'; wait for 20 ns; --
2,0 + 1,0 = 3,0
        a <= "01001100"; b <= "00111000"; sub <= '0'; wait for 20 ns; --
6,0 + 1,0 = 7,0
        a <= "01010000"; b <= "00111000"; sub <= '0'; wait for 20 ns; --
8,0 + 1,0 = 9,0
        a <= "00111100"; b <= "00111100"; sub <= '0'; wait for 20 ns; --
1,5 + 1,5 = 3,0
        a <= "00111010"; b <= "00111000"; sub <= '0'; wait for 20 ns; --
1,25 + 1,0 = 2,25
        a <= "01001100"; b <= "01000000"; sub <= '1'; wait for 20 ns; --
6,0 - 2,0 = 4,0
        a <= "01001100"; b <= "01001000"; sub <= '1'; wait for 20 ns; --
6,0 - 4,0 = 2,0
        a <= "01001100"; b <= "01001100"; sub <= '1'; wait for 20 ns; --
6,0 - 6,0 = 0
        a <= "00111000"; b <= "01001100"; sub <= '1'; wait for 20 ns; --
1,0 - 6,0 = -5,0
        a <= "01111110"; b <= "01111110"; sub <= '0'; wait for 20 ns; --
448 + 448 -> 448
```

```
        a <= "01001100"; b <= "00000000"; sub <= '0'; wait for 20 ns; --
6,0 + 0 = 6,0
        wait;
    end process;
end architecture sim;
```

tb_fp8_mac

```
library ieee;
use ieee.std_logic_1164.all;

-- Banc de test du MAC FP8 (E4M3)
entity tb_fp8_mac is
end entity tb_fp8_mac;

architecture sim of tb_fp8_mac is
    signal clk    : std_logic := '0';
    signal reset  : std_logic := '1';
    signal clear  : std_logic := '0';
    signal enable : std_logic := '0';
    signal sub    : std_logic := '0';
    signal a, b   : std_logic_vector(7 downto 0) := (others => '0');
    signal result : std_logic_vector(7 downto 0);
    signal valid  : std_logic;
begin
    uut : entity work.fp8_mac
        port map (clk => clk, reset => reset, clear => clear,
                  enable => enable, sub => sub, a => a, b => b,
                  result => result, valid => valid);

    clk <= not clk after 5 ns;   -- horloge 10 ns

    process
    begin
        reset <= '1'; enable <= '0'; wait for 12 ns;
        reset <= '0'; wait for 8 ns;

        -- Accumulation 1 : 1 -> 3 -> 9
        a <= "00111000"; b <= "00111000"; sub <= '0'; enable <= '1'; wait
for 10 ns;
        a <= "01000000"; b <= "00111000";                             wait
for 10 ns;
        a <= "01001100"; b <= "00111000";                             wait
for 10 ns;
        enable <= '0'; wait for 20 ns;

        -- Effacement puis Accumulation 2 : 4 -> 7
        clear <= '1'; wait for 10 ns;
        clear <= '0';
        a <= "01000000"; b <= "01000000"; enable <= '1'; wait for 10 ns;
        a <= "00111100"; b <= "01000000";                wait for 10 ns;
        enable <= '0'; wait for 20 ns;

        -- Soustraction : 7 - 2 = 5
        a <= "01000000"; b <= "00111000"; sub <= '1'; enable <= '1'; wait
for 10 ns;
        enable <= '0'; sub <= '0'; wait for 20 ns;
```

```
        wait;
    end process;
end architecture sim;
```